

面向长周期处理器开发的证据闭环 Agent Workflow

Loop-Work 四平面双闭环方法及 RV64 纵向案例研究

纵向工程证据范围：近四个月的持续开发记录

摘要：大语言模型已经能够生成寄存器传输级（Register Transfer Level, RTL）代码，并调用编译、仿真、综合与静态时序分析工具。随着本项目从 RV32 局部实现推进到 SoC、RV64 系统长跑和乱序性能优化，局部测试通过与工程交付之间出现了稳定差距：候选可以快速形成，但下游消费还要求设计与实验身份、判定器敏感性、长跑终态和性能资格同时闭合。本文研究的问题是，怎样把这种交付资格纳入 Agent 的工作过程，而不再把代码生成或命令成功直接视为完成。

本文提出 **Loop-Work**，将 Agent 的处理器开发过程组织为控制、知识、执行和证据四个责任平面。控制面冻结目标、权限和停止条件，知识面提供当前任务所需的有界规则，执行面产生候选与原始观测，证据面绑定实验身份并形成失败关闭的裁决。内层闭环处理同一任务中的定位、修改、回滚和重验；外层闭环把重复且可迁移的反例固化为版本化规则、程序化能力和验收协议。由此，Agent 的工程能力从“能否生成代码”转化为“能否在证据约束下形成可重复、可审计的交付”。

本文采用近四个月的单仓库纵向案例，覆盖 RV32/SoC/RV64 DiffTest、NPC/Verilator Ubuntu 22.04 系统事务、23 个可编译负向 RTL 变体，以及双发射乱序核的 CPI/PPA 资格化。约十九小时的系统事务到达预定终态，但原始 FAIL 仍被保留；冻结重放形成独立的版本化结论，owner-timing 与代理 PPA 在资格不足时分别保持 FAIL 和 UNQUALIFIED。这些材料表明四平面接口能够在不同验证压力下实例化，并使正结果、负结果和未资格状态进入同一证据链。本文结论限于该方法的可执行性与证据边界，不估计其相对其他 Agent 架构的生产率因果增益，也不以代理结果替代物理签核。

关键词：RTL Agent; Loop-Work; 领域知识; 证据闭环; 失败关闭; RV64; CPI/PPA

1 引言

1.1 问题背景与研究动机

早期 RV32 任务中，Agent 已经能够一次生成九个 RTL 文件并通过 Verilator lint；但同一结果没有 testbench、程序镜像、存储模型、退出协议或参考模型对拍。随后工程进入 SoC、RV64、OpenSBI、Linux、Ubuntu 和双发射乱序执行，单个候选开始同

时改变流水事务、异常顺序、设备模型、构建配置、workload 和验证判定器。约十九小时的系统事务更集中地暴露了这个矛盾：系统到达预定终态，但旧判定器使原始状态保持 FAIL；冻结日志可由版本化判定器重放，却不能覆盖原始状态；同一设计的代理 PPA 又因物理条件不足保持 UNQUALIFIED。因此，候选、证据解释和工程晋级是三个不同问题。它们共同支持一个较窄的起点：在本项目所观察阶段，候选生成周转已经快于验证与交付闭合。

这里的“快于”不是代码行/小时与验证行/小时的统计比较，而是过程性不匹配：候选可以在缺少动态验收时形成，而交付必须补齐身份、运行终态、判定器敏感性和晋级边界。若环境只向模型提供源码和 shell，模型可以迅速扩大修改范围，却无法自动判断：某条结果是否属于当前设计；局部测试是否覆盖目标错误；Linux 的最终症状对应哪个最早错误事务；一次综合成功能否承担 PPA 结论；判定器修改后应重跑十九小时系统，还是可以重放冻结日志。问题不在模型“会不会写”，而在模型是否处于一个能够定义边界、观察结果、反驳结论、恢复状态并安全停止的工程系统中。

由此，本文把核心问题确定为：如何为 Agent 产生的候选建立可被下一工程节点安全消费的资格，并使失败能够返回当前任务或沉淀为后续任务可复用的工程能力。基础模型决定候选搜索的上限，领域知识、受控执行和证据裁决决定候选能够到达的交付边界。

1.2 研究问题

全文围绕四个研究问题展开：

1. **RQ1（闭合性）**：怎样把自然语言中的“完成”转换为可执行合同和可验收交付单元？
2. **RQ2（拒绝能力）**：怎样区分 RTL 缺陷、身份漂移、判定器缺陷、量测不合格和运行不完整，并阻止它们形成假 PASS？
3. **RQ3（尺度扩展）**：同一方法能否在局部 RTL、SoC、RV64、操作系统长跑和乱序微架构等不同压力层级上保持同一接口可实例化？
4. **RQ4（多目标资格化）**：在功能正确性不可放松的前提下，怎样开展 CPI/PPA 诊断而不把代理指标越级为晋级结论？

1.3 主要贡献与研究范围

本文围绕上述问题给出四项贡献：

1. 将权限、知识适用性、真实执行和证据资格形式化为控制、知识、执行和证据四类责任状态，并以最小不可区分反例检验这些状态能否被无损省略。

2. 给出任务内交付和跨任务能力演进两类反馈边，以及身份拒绝、负向资格、长跑失败关闭和有界晋级四条可证伪命题（P1–P4）。
3. 给出由版本化知识、类型化合同、受控执行、证据收据和有界历史构成的仓库级参考实现，说明四平面接口如何进入真实开发环境。
4. 给出 RV32/RV64、single/SoC、NEMU/NPC、Ubuntu 22.04 长跑和双发射 OoO CPI/PPA 的纵向案例，并以 30 条机器可读主张和独立反例复核综合正结果、负结果与未资格状态。

研究材料来自一个持续演化的仓库，本文据此讨论方法接口的可执行性、可审计性及其在所列 RV64 压力层级上的实例化，不把纵向共现解释为生产率因果效应。全文依次讨论相关工作、问题与设计要求、方法与参考实现、案例研究设计、纵向工程证据、命题与成果综合、未闭合目标及结论。

2 相关工作与方法空缺

2.1 Agent、RTL 生成与 EDA 自动化

SWE-agent 说明 Agent–Computer Interface 会影响仓库级软件任务的执行方式 [1]；VerilogEval 用 testbench 通过率评价 Verilog 生成 [2]；ChatEDA 研究大模型对 EDA 工具链的规划与执行 [3]。MAGE 进一步用多 Agent 候选采样、调试和 Verilog 状态检查提高模块级生成结果 [13]。近期一项 CVDP 实验显示：对多数受测模型，直接套用基线工具 Agent 的 Pass@1 低于相应的无工具单次基线；加入结构化系统提示后，差距缩小，部分配置持平或反超 [14]。这一结论只适用于该论文模型、任务子集与 harness，不说明 Agent 在一般 RTL 任务中更弱。NotSoTiny 进一步在结构更复杂的上下文模块补全任务上比较原生 testbench 与形式等价：在 1114 个去重并验证的模块补全任务上，语法有效输出有 95–97% 可通过原生 testbench，而 11 个受测模型在形式等价下的完全正确率为 3.3–23.4%；其中四个相对较高结果也仅为 19.0–23.4% [22]。其中时序设计采用有限深度的 `equiv_induct` ($k = 30$)，因而这一范围只描述其数据集、模型和验证协议，却直接提醒读者：正向 testbench 接受率不能替代独立的语义资格。

ACE-RTL 以 generator、reflector 和 coordinator 维护迭代调试上下文，其预印本报告在 CVDP 上相对 14 个基线最高 41.02% 的 pass-rate 改善 [19]；NVIDIA 随后的官方技术发布又报告 Nemotron 3 Ultra 配合该 Agent 在九类 CVDP 任务上的平均 pass rate 为 97.1% [20]。本文以前一项预印本结果作为学术论证锚点；后一个数值来自未经同行评议的厂商技术博客，是平均通过率而非 Pass@1，只作补充背景，也不能延伸为系统级签核率。FluxBench 在统一提示、工具和工艺库下评价 RTL 生成与 RTL-

to-GDS 流程，报告同一基础模型的 Agent 系统在固定 DeepSeek-V4 与 350 MHz 条件下的 0–100 端到端得分为 11.67 至 97.94，即相差 86.27 个绝对分值点；这不是 86.27% 的相对提升。其当前 v3 在 PicoRV32、55 nm、350 MHz、各配置仅运行一次的实验中，还给出两项端到端得分同为 13.33、但归一化 Token ROI 为 12.71 与 0.09 的 Agent 配置，二者相差约 141.2 倍 [21]；这不是纯 Token 数之比，也不能归因于单一模型或单一组件。Design Conductor 则报告从 219 词需求出发，在 12 小时内为 RV32I+Zmmul VerCore 形成满足 ASAP7 1.48 GHz 时序的 GDSII[25]。这些结果表明，相关系统已在各自任务与评价协议下报告了迭代反馈和端到端自治能力的进展；由于设计、工具、工艺与 oracle 均不同，它们不构成本文 RV64 OoO、Ubuntu 长跑或 PPA 资格的横向基线。

2.2 仓库级修复与 Agentic 验证

CVDP 将 RTL 生成、验证、调试和规格对齐组织为 783 个问题及 13 类任务 [15]；HWE-Bench 把真实历史硬件缺陷转化为六个开源项目中的 417 个仓库级修复任务，并指出定位、硬件语义与 RTL/配置/验证跨产物协调是主要失败环节 [16]。PRO-V-R1 使用程序化工具反馈训练自主 RTL 验证 Agent[17]；GoGoTB 则把执行控制、可演化知识和规格绑定覆盖闭合连接起来 [18]。这些工作已经覆盖角色分工、反馈循环、验证环境生成和仓库任务，因此本文不以“首次使用 Agent 闭环”为贡献。本文关注的是另一条轴：同一 repository-native 生命周期中，怎样把任务权限、适用知识、真实执行和证据资格保持为可区分状态，并让中断、旧身份、弱 oracle 和不合格 PPA 进入不同终态。

LLM4Cov 把确定性执行反馈用于离线 Agent 学习，并在 CVDP-ECov 的 Agentic evaluation 上报告：4B 模型生成测试平台的 pass rate 为 69.2%，平均覆盖率为 90.4%[26]；二者都不是 RTL 功能正确率或完整验证闭合率。Si2 的 LBC-bench 则以 CVDP 为初始重点，公开 leaderboard 数据格式、提交标签和可复现入口 [28]。前者研究怎样提高覆盖闭合候选，后者提供公共比较基础设施；二者都可为后续对照实验提供任务或评价协议，但公开 leaderboard 本身不能充当隐藏 oracle，也不承担本文主张的独立真值责任。

2.3 Mutation 与硬件故障注入

Mantra 从真实硬件缺陷构造 mutation 算子，并以 mutation score 评价测试敏感度 [4]；基于 HDL 的故障注入研究也表明覆盖率受到故障模型、工作负载和注入技术影响 [5]。Firefly 在 Maveric Core 2 与 Bitty 两个对象上报告：组合 line/toggle coverage 分别达到 95% 与 90% 时，其注入 fault set 中仍分别有 28% 与 67% 未被检出 [23]。该结果不能外推到所有覆盖率或所有故障模型，但说明常规覆盖率与 fault sensitivity 不是

同一构念。BugGen 则用多 Agent 生成、插入和验证 RTL 缺陷，在五个 OpenTitan IP 上产生 500 个唯一注入缺陷，报告 94% functional accuracy 与每小时 17.7 个 validated bugs；其中 104 个生成变体绕过既有回归 [24]。这里的 104 是未被回归检出的注入变体，不是 104 个新发现的 OpenTitan 真实缺陷。

Firefly 与 BugGen 主要解决上下文相关 fault 的生成、验证基础设施诊断或数据集扩展；本文后述的负向资格化命题关注这些 fault 如何成为不可绕过的晋级资格。为此，“变体可编译、语义变化非空、错误被触发、错误传播到观察点、判定器拒绝、campaign 完整”必须同时成立。本文当前 23 个定向 mutant 与上述论文的数据集没有共同分母；检测给定变体仍然不能证明 DUT 全局正确或 oracle 完备。

2.4 追踪、可信论证与构建来源

需求追踪研究关注需求来源和生命周期链接 [6]。W3C PROV-DM 以 entity、activity、agent 及其派生关系表达来源与责任 [29]；GSN 和 SACM 则组织 goal/claim、argument、context 与 evidence 的可信论证 [30, 7]。in-toto 与 SLISA 强调供应链步骤、materials/products、构建来源和抗篡改控制 [8, 9]。这些标准能够承载本文的部分对象，但来源真实不等于证明力充分，论证结构完整也不等于仓库作业已经执行。本文不发明 provenance 或 assurance-case 语法，而把它们与 task contract、工程 oracle、失败关闭状态和晋级责任连接；这里的 oracle 是把日志、trace、计数器或波形转换为机器判定的参考模型、断言或规则，它本身也必须被资格化。

2.5 质量门、UVM 与工业准备度

持续交付中的 quality gate 研究讨论发布可信度与速度的权衡 [10]；UVM 标准化验证组件和接口 [11]；ISO 26262-5 规定汽车硬件安全生命周期和工作产品 [12]。Siemens 与 NVIDIA 的 2026 年官方产业发布把长运行 Agent、OpenShell 的 access control/audit trail 与确定性、physics-based EDA engine 的持续校验组合为 self-verifying workflow[27]。这说明工业系统也在把验证反馈和治理放入 Agent 运行时；该公开材料是产品/技术发布，没有给出可用于因果比较的同行评议实验，也未把所有责任状态形式化为本文的类型化晋级序列。

本文不声称符合 UVM、SLISA 或 ISO 26262，也不替代物理 signoff。本文将上述已有机制组合并约束到 repository-native 场景：把本地 module/signal/transaction、验证层级、实验身份、负向判定器能力和失败关闭发布编译成同一个可执行工作流。

表 1: 相关方向与本文方法的接口

方向	主要回答	本文补充的问题
Agent/RTL benchmark	候选是否通过给定任务或 testbench	将模块和仓库任务结果绑定到限定声明
Repository/verification Agent	如何修复真实缺陷、生成验证环境或闭合覆盖	长跑、身份和发布状态怎样失败关闭
Self-verifying EDA stack	物理引擎反馈、治理运行时和模型怎样进入自治执行	身份、新鲜度、负向敏感性、失败关闭与成熟度如何组成不可绕过的资格序列
Public leaderboard	模型与 Agent 提交怎样透明比较和复现	公开分数与隐藏 oracle、下游消费资格怎样分离
Mutation/fault injection	给定 fault set 下测试能否观察错误	变体非空性、传播、拒绝和 campaign 终态
追踪、来源与可信论证	claim、活动、来源、论据和证据如何关联	来源与结构之外，证据是否新鲜且足以支持声明
Quality gate/UVM	验证环境复用与发布控制	Agent 候选如何按本地工程合同失败关闭
Signoff readiness	何种物理证据可以交付	代理统计为何不能越级为合格 PPA

文献检索截至 2026 年 8 月 5 日；正式会议论文、camera-ready 作者稿、arXiv 预印本和产业技术发布在参考文献中分别标明。不同来源共同用于确定研究问题的位置，本文的方法命题仍由后续仓库证据和专门实验承担检验责任。

综上，近期工作已经把 RTL 智能开发推进到带仿真反馈、工具调用、状态保持和阶段门禁的长程流程，但候选何时具备下游消费资格仍缺少统一接口。本文据此把身份、新鲜度、负向敏感性、失败关闭和有界晋级组合为仓库原生的交付协议。下一章先从处理器工程失效模式推导这套协议必须表达的责任状态。

3 问题定义与设计要

相关工作表明，候选生成、工具反馈、来源追踪和阶段门禁已经分别取得进展，但它们尚未统一回答：一个候选在何种任务身份、知识边界、执行状态和证据条件下，才可以被下一工程节点消费。本文先把这一缺口写成仓库级交付问题，再由失效模式推导方法所需的责任接口。

3.1 处理器 Agent 任务的失效结构

处理器开发任务不能只写成“修改某个模块”。本文把任务定义为

$$\mathcal{T} = \langle c, O, K, U, Q, E \rangle,$$

其中 c 是待证明的工程主张, O 是 RTL、软件、配置与 workload 构成的对象集合, K 是必须遵守的领域知识, U 是授权动作, Q 是分层验收谓词与门禁, E 是必须保留的证据。开放式 Agent 往往只接收 c 的自然语言近似和部分 O , 因而在对象、动作和完成条件上留下歧义。任务还必须绑定实现责任与独立复核责任; 具体角色接口在控制面小节统一定义。表 2 从实际工程压力推导出方法要求。

表 2: 处理器工程失效模式与方法要求

工程困难	开放式 Agent 的典型失效	方法要求	Loop-Work 机制
跨模块事务耦合	局部修改通过, 却破坏 owner、flush、异常序或总线生命周期	最小充分且可扩展的任务边界	合同编译、专业不变量与范围扩展协议
target/reference 多配置	把另一设计、另一配置或旧日志当作当前结果	比较前绑定完整实验身份	design、build、workload、oracle 与 evidence 分层身份
小时级系统运行	进程退出、前台超时或信号中断被写成成功	成功必须由证据末端授权	失败关闭证据提交协议
CPI/PPA 多目标优化	用局部 CPI、generic cell 或理想时钟结果交换功能正确性	正确性硬门先于性能比较	分层门禁、单机制候选和成熟度晋级
多会话历史增长	无关历史淹没当前问题, 过期结论被重复消费	最小充分、带来源的历史选择	retained knowledge、evidence index 与 bounded recall
多 Agent 协作	节点越界写入、上下文污染或实现者自证完成	明确 owner、作用域与独立审查	机器合同、隔离派发与 reviewer/inspector

这些失效并不是六个彼此独立的脚本问题。它们可进一步归并为四类不可互相替代的责任缺口: 目标和权限没有被控制; 规则虽存在却没有被选择到当前任务; 命令虽然执行却没有形成受约束的真实观测; 观测虽然存在却没有获得足以支持主张的资格。若把四类状态压成一个 GREEN, 身份漂移、弱判定器和不完整长跑就会变成不可表达状态。

3.2 由失效模式推导的设计原则

上述四类责任缺口导出六项设计原则:

1. 合同先于生成: 先定义对象、输入、输出和成功条件, 再允许 Agent 修改或运行。

2. 身份先于比较：baseline 与 candidate 身份不一致时，不计算正式改善。
3. 原始观测不可改写：新 checker 可以生成新解释，但不能覆盖原始 FAIL/PASS。
4. 正向与负向证据并存：当前版本 PASS 还要接受能够暴露目标错误的反例检查。
5. 局部接受与工程晋级分离：focused、module、system、PPA 分别承担不同结论。
6. 高成本验证受决策约束：长跑必须能够改变明确决策，并在开始前完成低成本排除。

这些原则共同要求一种带类型的闭环：控制责任限定任务，知识责任提供适用规则，执行责任产生候选和原始观测，证据责任决定这些观测能够支持什么结论。增加工具数量只能扩大动作集合；只有四类责任都可以独立检查，扩大的动作集合才会转化为更强的交付能力。

3.3 术语的操作性定义

为使上述责任可以被复核，表 3 给出本文自定义术语的可执行含义。

表 3:本文术语及其操作性定义

术语	定义
完成主张 (claim)	对功能、系统行为、性能或成熟度的陈述，例如“该 design 的 FENCE 顺序合同闭合”。
证据边界	某组设计、配置、输入、命令、oracle 和原始产物最多能够支持的结论。
oracle / 判定器	把日志、trace、计数器或波形转换为 PASS/FAIL/GAP 的规则；它本身也需要资格化。
Loop-Work	从冻结合同到真实观测、差异分类、下一动作和可恢复终态的一次闭环工作轮。
task-run	一次确定性工程事件的结果档案；保存状态、指针、哈希和有界日志。
bounded recall	只按任务 profile、规范和关键词加载必要历史，避免过期结论淹没当前问题。
design-id	production RTL 规范文件集合的内容哈希，用于区分设计身份。
mutation	能完成编译或 elaboration 的负向 RTL 版本，用于检查 testbench/oracle 是否拒绝指定错误。

术语	定义
owner	某个流水事务或状态在当前周期的唯一责任主体，用于阻止旧槽位或错误生产者继续授权结果。
fail-closed	身份未知、证据缺失、清理失败、信号中断或 oracle 不合格时保持非 PASS。
promotion / 晋级	候选从局部开发证据进入更高成熟状态；它需要满足父级门禁。
CPI / IPC	每条退休指令的周期数 / 每周期退休指令数；必须绑定测量窗口和退休语义。
PPA	性能、功耗、面积（Performance, Power, Area）的联合工程指标；其资格取决于实现条件。
Skill / 程序化任务能力	把专业规则编译为可重复步骤、参数、模板和机器合同的程序。
E2E / 端到端闭包	End-to-End 的缩写；把规则、Skill、执行入口、观测点和终止条件连接为可运行任务图。
profile / 执行剖面	一类任务的节点、依赖、实现函数、责任主体、输入和输出清单。
raw observation / 原始观测	工具未经结论化处理的日志、trace、波形、返回码和运行状态。
manifest / 清单	对设计、配置、命令、产物路径、尺寸和哈希的结构化登记。
receipt / 收据	对已资格化证据及其允许结论的机器可读记录。
replay / 冻结重放	在设计、输入和原始观测不变时，用版本化判定器重新解释同一证据。
gate / 门禁	由固定验收谓词和执行入口组成的晋级检查。

3.4 声明与分层实验身份

只使用 design-id 会遗漏 testbench、工具、workload 和判定器的变化。因此一次强结论至少由表 4 中的身份共同限定。

checker-only 修改若没有改变 DUT 输入、时序、终止或随机性，并且原始输入与日志完整，可以对冻结证据做 versioned replay（版本化重放）；production RTL、设备模型、仿真语义或 workload 发生变化时，旧执行不再属于新对象。该规则既防止串证，也允许十九小时级原始观测在身份不变时接受新的判定器解释。

表 4: 分层实验身份

身份	绑定对象
design-id	production RTL 规范文件集合及参数化设计点
verification-id	testbench、assertion、reference、oracle 与 mutation 规则
build-id	filelist、define、工具版本、编译、elaboration 与链接配置
workload-id	镜像、binary、输入、ROI、seed、设备与延迟配置
experiment-id	上述身份与执行语义的组合身份
evidence-id	原始日志、结构化结果、状态和发布产物的内容哈希

当失效模式被归结为目标失真、知识失配、执行越界和证据误认后，所需方法便不再是增加一次生成，而是为这四类责任建立彼此可检查又能闭环协作的接口；它们构成 Loop-Work 顶层结构与可证伪预测的来源。

4 Loop-Work 顶层方法

4.1 中心论点：从生成候选到授予消费资格

因此，本文不再把问题表述为“怎样让 Agent 生成更多 RTL”，而表述为：怎样使一个候选获得可被下一工程节点安全消费的资格。本文所称 Agent，是能够读取仓库材料、调用工具并根据运行结果继续行动的大语言模型执行单元；Workflow 是带输入、输出、权限、成功条件、失败分流和证据责任的可执行任务图；Loop-Work（闭环工作轮）则要求一次有效迭代从明确合同出发，经真实执行和证据裁决，以可恢复状态结束。一次聊天往返、一次命令返回 0 或一行 PASS 文本都不自动构成闭合工作轮。

本文的中心论点是：Agent 的工程能力不由一次生成质量定义，而由可发现知识、授权执行、身份绑定证据和失败关闭裁决所共同形成的可重复闭环定义。Loop-Work 把原本隐含在工程师经验、角色约定和脚本中的责任转化为可版本化、可审计、可证伪的状态与接口。闭合、拒绝、尺度和多目标资格化四个研究问题分别检验该论点在任务定义、错误阻断、压力扩展和性能晋级上的含义。

4.2 抽象顶层：四类责任状态与两条反馈边

Loop-Work 的根输入是工程目标 G 、当前状态 S 、约束边界 Γ 与验收准则 A 。控制面把根输入编译为合同、权限、owner、门禁和终止条件；知识面返回当前节点适用的规则、事实和程序化能力；执行面在授权范围内产生候选与原始观测；证据面绑定实验身份、判定器和证据资格；控制面最后作出接受、返工、保持缺口或终止的裁决。其责任状态与裁决写为

$$s = \langle C, K, X, V \rangle, \quad d(s) \in \{\text{PASS}, \text{FAIL}, \text{GAP}, \text{UNQUALIFIED}, \text{BLOCKED}\}.$$

这里的 C 是控制责任， K 是知识适用性， X 是实际候选与运行状态， V 是身份绑定、oracle 与证据资格。图 1 给出全文唯一的动态总览： C_{prepare} 和 C_{decide} 是同一控制面的准备端与裁决端， $K/X/V$ 分别对应另外三个平面。内层闭环把当前任务的反例和证据缺口送回当前合同，外层闭环只把重复且可迁移的缺口送回可复用能力。双闭环是四平面主干上的两类反馈边，不是第五和第六个平面。

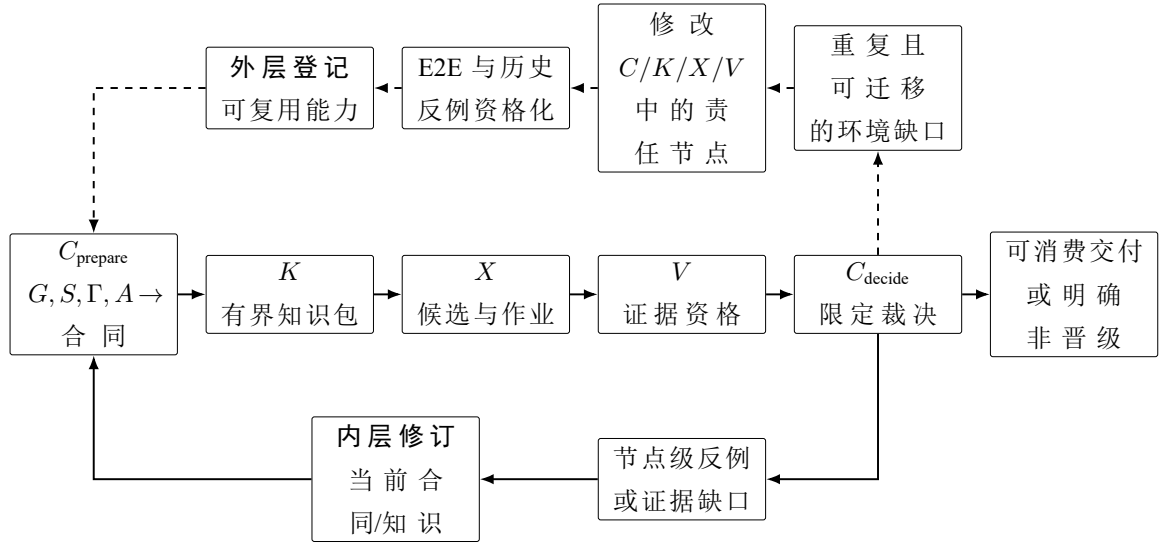


图 1: Loop-Work 顶层方法：四类责任状态沿主干流动，内外闭环以不同半径返回

上述结构给出一条总纲：候选只有依次获得权限、知识适用性、真实执行和证据资格，才能进入下游；任一状态不足都保留为 FAIL、GAP、UNQUALIFIED 或 BLOCKED。内层闭环面向当前任务的交付，外层闭环面向可供未来任务复用的能力候选；二者共用四平面状态，但具有不同的返回半径和晋级条件。

4.3 可证伪预测与方法边界

Loop-Work 对四类关键失效给出表 5 中的预测。它们是可以被反例推翻的接口命题，而不是由参考实现的存在自动成立的效果声明。后文先说明各责任面的形式化接口和工程实例化，再用案例材料报告这些预测当前得到何种支持。

四条预测回答“在给定接口条件下应当观察到什么”；它们不直接估计相对其他 Agent 架构的生产率。这个区分把方法命题、参考实现和案例证据放在同一条推理链上，又使三者承担不同责任。

表 5: Loop-Work 的四条可证伪预测

命题	方法预测	必要条件	直接证伪事件
P1 身份拒绝	不同实验身份的观测不得支持当前候选	design、verification、build、workload 与 evidence 身份可核验	身份不匹配仍形成可消费 PASS
P2 负向资格	正向通过还需证明判定器能够拒绝指定非等价错误	mutant 可编译、非空、被触发并传播到观察点	合格 mutant 存活而结果仍获晋级
P3 失败关闭	长跑只有在终态 marker、命令、清理和信号状态共同闭合后才能发布 PASS	状态提交与证据发布不可分割	中断或证据不完整仍遗留可消费 PASS
P4 有界晋级	局部结果受父级门禁约束，跨任务能力受复用和稳定阈值约束	回滚基线、父级 gate 与外层阈值均被版本化	局部失败越级，或未达阈值的能力被标记为已晋级

5 Loop-Work 四平面的形式化与职责边界

5.1 静态职责树与最小充分性

在第 4 节的动态总览基础上，本章把同一方法拆成静态职责树、平面合同和最小反例，回答每一类状态的输入、输出、权限边界以及为何不能被无类型 GREEN 位替代。

在第 4 节定义的根输入和 $C/K/X/V$ 主干上，图 2 把动态序列投影为静态职责树：控制面是生命周期编排根，知识、执行和证据面以类型化接口接受其请求。该图只表达从属和职责分解，不表达时间先后。

表 6: 四个面的输入、输出与不可越过的职责边界

平面	主要输入	类型化输出	本面无权完成的动作
控制面	G, S, Γ, A 与上一节点状态	任务类别、节点合同、owner、门禁和终止决定	不替代专业事实、工具执行或证据判定
知识面	当前节点查询、工程作用域与来源要求	有界上下文包、专业不变量和可调用程序	不直接修改设计，也不以文档存在宣称规则已经执行
执行面	节点合同、知识包、允许命令和输入身份	候选变更、返回码、raw log/trace/report 与运行状态	不根据退出码自行宣称工程 PASS
证据面	原始观测、身份、oracle、fault set 与验收谓词	PASS、FAIL、GAP、UNQUALIFIED、BLOCKED、receipt 与索引	不改写原始事实，不越过未闭合门禁授权晋级

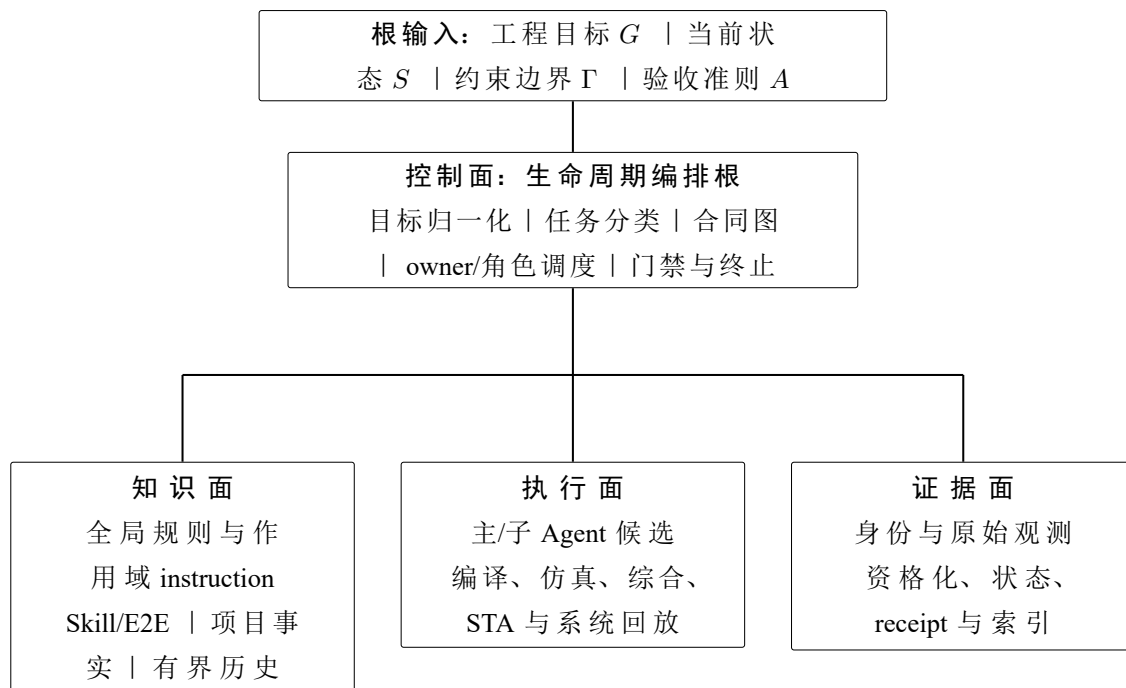


图 2: Loop-Work 的树状职责架构; 返回接口与运行顺序在图 5 中给出

四平面不是四个必须独立部署的进程或目录，而是裁决函数必须分别观察的四类责任状态。对第 4 节定义的 $s = \langle C, K, X, V \rangle$ ，若省略某一坐标后存在两个投影相同而正确裁决不同的状态，则该坐标不能被无损省略。表 7 给出本案例中的最小状态对。

表 7: 省略一个责任状态后出现的最小不可区分反例

被省略状态	其余状态相同的两个事件	正确裁决为何不同	单一既有机制不足之处
控制 <i>C</i>	同一源码、规则与证据；一次是只读 review，一次是获准修改的 development	前者写入即越权，后者必须产生变更与验证证据	ACI 能调用工具，但不自动给出当前 owner、写权限与终止合同
知识 <i>K</i>	同一历史 PASS 来源真实；一个合同仍适用旧接口，另一个合同已更换配置/事务语义	前者可引用，后者只能保持 GAP 并补充当前规则	provenance 能证明“曾由何物产生”，不自动证明“现合同仍适用”
执行 <i>X</i>	同一合同与判据；一项作业到达证据末端，另一项被 TERM 中断且 marker 缺失	前者可进入资格化，后者必须 FAIL	assurance case 或静态 gate 可组织主张，却不自动执行进程和清理状态机
证据 <i>V</i>	两个命令均返回 0；一个绑定当前身份并能拒绝目标 mutation，另一个消费旧日志或弱 oracle	前者可限定 PASS，后者只能 GAP/UNQUALIFIED	quality gate 可算谓词，若未绑定身份和负向敏感性仍会产生假绿

因此，本文的必要性主张是“状态类型不可无损省略”，而不是“必须建立四套工具”。ACI、quality gate、W3C PROV、GSN/SACM 可以分别实现这些接口的一部分；若一个实现把它们合并到同一数据库或程序，但仍保留四类字段、版本和裁决边界，则只是物理封装发生变化，并没有消除这四类责任。反之，若合并结果只剩一行无类型 GREEN，表 7 中的状态对将不可表达。

这一分解也不与最新 Agent 架构或产业 self-verifying stack 争夺同一层次。ACE-RTL 的 generator、reflector、coordinator 是候选生成和调试上下文的角色分解 [19]；Siemens 公开材料列出的确定性 EDA engine、NeMo Gym、OpenShell、Nemotron/Switchyard 与 GPU 加速是技术栈构件 [27]。前者可承载 *K/X* 的迭代，后者可分别承载执行、治理和判定接口，但这些物理构件怎样部署并不自动决定裁决状态怎样编码。就公开材料而言，它们没有报告与本文相同的 design/build/workload/oracle/evidence 强身份元组、mutation 资格和成熟度全序；这里应理解为“未报告同一形式化接口”，而不是断言其系统不存在等价内部机制。反过来，Loop-Work 的四面也不替代专用模型、物理引擎或安全运行时；它只规定这些构件的输出在成为下游结论前必须保留哪些可区分责任状态。

本文的贡献定位于责任状态的显式形式化，而非新增底层工具能力：权限、知识适用性、真实执行和证据资格在既有系统中可能由不同工具隐式承担，Loop-Work 的增量是把它们编码为可版本化、可审计且可证伪的责任状态。若这些状态未被类型化，表 7 中“观测相同而正确裁决不同”的状态对便无法区分，责任归属、旧证据失效和失败关闭消费也无法由第三方复核；显式形式化则为 P1–P4 提供了可检查的接口。该收益来自可表达性与可审计性，而不是对既有工具能力的优先权声明。

目标进入体系后，首先必须确定它属于哪类工程活动、允许改变什么以及在何种条件下结束，因此先从控制面展开这棵树。

5.2 控制面：从根目标到权限化任务图

控制面回答四个问题：本轮究竟要证明什么，谁拥有当前节点，允许改变哪些对象，以及哪些结果能够终止节点。它先把自然语言请求归一化为任务类别，再把类别映射为权限、证据义务、收尾门禁与留存档位。因此，分类不是给任务贴标签，而是一个全定义路由函数

$$\mathcal{R}(\tau) = \langle W_\tau, E_\tau, Q_\tau, L_\tau \rangle,$$

其中 τ 为任务类别， W_τ 为写入权限， E_τ 为最低证据义务， Q_τ 为终止时允许或必须执行的门禁， L_τ 为 task-run 留存级别。每一种类别都必须得到四个值；“零门禁”和“禁止写入”也是明确的路由结果，并非该类别没有经过控制面。

表 8: 任务类别对写入权限、证据、门禁和留存的完整映射

类别	W_τ : 允许写入	E_τ : 最低证据	Q_τ : 终止门禁	L_τ
review	禁止; 只读源码、spec 与证据	可定位的审查发现、反例或明确无问题范围	零门禁; 结论本身即交付	none
analysis	禁止; 只读研究	调用链、数据流、状态或根因的来源指针	零门禁; 不得借分析身份落盘	none
docs	仅普通文档与文档资产	来源锚点; 可构建文档还需编译与渲染检查	不自动派生业务门; 显式文档门可选	none
development	合同内 RTL、C/C++、脚本或软件	变更指针及至少一项真实开发验证	实现阶段执行受影响 domain test; 不自动追加环境 E2E	compact
verification	禁止修改 production 源码; 可产生运行证据	配置、命令、返回码、日志及 PASS/GAP 边界	要求指定验证 evidence; 不追加环境门	none
environment	AI 规则、Skill、profile、合同与维护工具	定向自测、反例及路径对应的环境证据	只运行显式路径映射到的固定 gate pointer	compact
longrun	合同内 runner/配置; 运行中不混入未登记改动	design/config identity、marker、哈希、signal、cleanup	必须由 fail-closed 状态协议授权 PASS	durable
cleanup	仅精确列出的缓存、编译物或归档目标	删除前 preview 与删除后 result 清单	目标路径复核; 涉及索引卫生时运行对应门	compact
release	发布合同授权的源码、清单与包	完整 manifest、门禁结果和独立复核边界	release 门禁; 失败不得形成正式发布	durable

表 8 也解释了“分类决定是否写源码、需要何种证据”之间的因果关系: review/analysis 的决定是禁止写入且不增加实现门; development 的决定是允许受限写入但必须已有真实业务验证; verification 的决定是保持 production 源码只读并提交运行证据; environment、longrun 和 release 则分别增加环境指针、失败关闭或发布门禁。若用户扩大范围导致这些值改变, 控制面必须重新分类, 不能沿用旧类别的权限继续执行。

5.2.1 “大目标”与图节点的关系

本文所称大目标并非原始对话全文, 而是归一化后的复合验收目标

$$G = \langle \text{claim}, \text{objects}, \text{hard constraints}, \text{acceptance} \rangle.$$

例如，一次 RV64 性能基线复核的大目标是：在同一设计、配置和 workload 身份下，核算双发射计数器端点与 cycle/slot 守恒，检查多次运行和统计开关的非干扰性，保留 A1（第一次原始性能基线执行）和 A2（同一冻结配置下的第二次独立执行）的原始 FAIL，并在物理条件不足时维持 PPA 为 UNQUALIFIED。该目标同时包含多个可独立失败的义务，不能交给一个“查看全部然后判断”的超大节点。控制面因此把它拆成依赖图：身份冻结、计数器核算、重复性检查、失败状态复核、独立反例审查和聚合裁决分别成为节点。

图节点不是一段伪代码，而是一个具有输入端口、授权边界和输出端口的合同，如图 3 所示。上游只能提供合同列出的输入；节点内部动作受 owner 与权限包络约束；下游只能消费已经声明的产物和证据。缺少 evidence 的节点不能成为下游硬依赖。

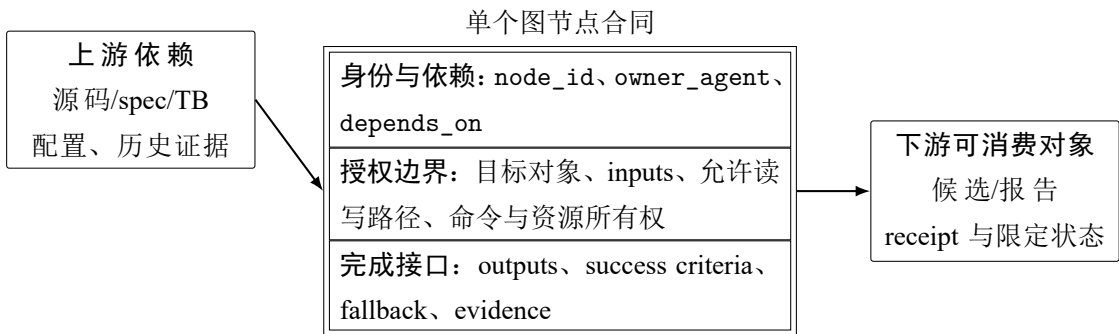


图 3: 图节点的最小合同：依赖、权限和完成接口属于同一个可审计对象

5.2.2 调度器、实现者、审查者与 oracle

在前述任务定义与设计要求的基础上，控制面进一步固定以下四类角色及其接口：

1. **调度器**：维护节点依赖、读写权限、owner 分配与状态迁移；它决定哪个节点可以开始，但不替代节点内的专业判断。
2. **实现者**：在合同授权范围内生成候选，并运行合同指定的定向验证；它提交候选和原始观测，无权用自己的总结宣布独立审查通过。
3. **审查者**：作为独立只读节点，不继承实现者的结论，依据同一验收谓词寻找反例、转录错误和越级声明；其 PASS 只覆盖合同列出的材料，不能替代未运行的 testbench、综合或 STA。
4. **判定基准 (oracle)**：由参考模型、断言、checker 或形式化谓词提供机器可判定的比较标准。它是验收接口而不是人格角色，也不能在输入身份不一致时给出有效裁决。

控制面至此给出了问题、权限、角色和终止条件，但它本身不提供 RTL 事实、专业规则与历史经验；这些输入由知识面按当前节点构造。

5.3 知识面：从开放仓库到有界上下文包

知识面回答“当前节点需要知道什么、这些内容从何而来、哪些结论允许复用”。其输出不是文件名列表，而是带来源的有界上下文包

$$B_i = \langle R_i, F_i, H_i, P_i, \Pi_i \rangle,$$

其中 R_i 是不可违反的规则， F_i 是当前源码、spec、testbench、配置和工具事实， H_i 是按当前 profile 与主题检出的历史证据， P_i 是可调用的 Skill、E2E 与 oracle， Π_i 是每一项内容的路径、版本或哈希来源。控制面以节点合同为查询条件，知识面只返回完成节点所需的最小充分集合；全量聊天、无关日志和过期快照不进入 B_i 。

表 9: 知识面的层级、输出语义及其边界

知识层级	回答的问题	进入节点的形式	单独不能证明什么
全局 policy/AGENTS 作用域 instruction/spec 项目事实	所有任务都不能越过哪些 行为边界 当前模块、接口或 PPA 路 径必须保持什么不变量 当前设计、配置、TB、工 具与 workload 实际是什么	任务分类、语言、记录和 安全约束 硬约束、验证顺序、接口 谓词与晋级条件 合同允许路径内的源码片 段和身份元数据	不能替代模块 spec 或业务测试 文件存在不等于 不变量已被执行 当前事实不自动 成为跨任务稳定 知识
Skill	某类任务应当怎样重复执 行	参数化步骤、合同生成器、 固定命令目录与反例出口	Skill 自测不等于 被测设计正确
E2E	哪些规则、Skill、执行入 口和观测点必须连成闭包	执行剖面的依赖、节点函 数、输入输出和终端条件	任务图可展开不 等于业务目标已 经通过
oracle / 判定器	原始观测满足什么条件才 可形成限定结论	比较器、断言、目标标记 和裁决规则	给定判定器通过 不等于未建模错 误均可见
bounded recall	哪些已发布历史与当前节 点相关	受 profile、主题和预算限 制的摘要及 evidence 指针	历史 PASS 不能 授权身份已经变 化的新候选

这里需要区分 instruction、Skill 和 E2E。instruction 是声明式规则，规定“什么不能错”；Skill 不是知识本身，而是把已经选定的专业规则编译为可重复执行的步骤、

合同和产物模板；E2E 也不是“完成知识”，而是把规则、Skill、执行入口、观测点和终止条件组织为可执行闭包，并检查这些对象之间的接线与证据链。控制面依据任务类别与路径选择三者，执行面调用其中的操作，证据面消费其中的验收谓词。因而 Skill/E2E 虽以文件实现，却不是与四面平行的第五层：它们首先属于知识面，只有被控制面选择后才在执行面运行，并由证据面裁决。

以本地 RV64 子任务为例，`rtl-agent-task-contract.instructions.md` 没有保存某次对话答案，而是把“审查一段 RTL”拆为工程域、任务类型、可读路径、可写路径、规范命令、必读 RTL/spec/TB、交付物和成功条件。相应 Skill 将这些字段编译为带版本与 SHA-256 的 JSON 合同；E2E 再检查 instruction、Skill、合同生成器、执行节点与证据入口是否接通。这样，一条接口经验才从供模型阅读的文字，转化为能够限制写入、拒绝越界命令并接受反例检验的工程能力；合同检查通过仍只证明任务边界清楚，不能替代该 RTL 的 testbench、DiffTest、综合、静态时序或 PPA 判定。

知识面还承担外部知识进入工程的净化边界。一次 task-run 的原始日志只进入证据索引；被反复验证且跨任务稳定的结论才进入 retained memory；需要持续约束动作的结论进入 instruction；可重复操作进入 Skill；需要自动组合和验收的能力才进入 E2E。这样，工作区不会把“发生过”误写成“以后都成立”，也不会把所有历史过程重新灌入模型。

有界上下文包把开放工程目录压缩为当前节点能够消费的确切输入，执行面据此产生候选和原始观测。

5.4 执行面：在合同授权内产生候选与原始观测

执行面只回答“实际做了什么”。它接收节点合同和有界上下文包，调用主 Agent、子 Agent、编译器、仿真器、DiffTest、综合、STA 或系统回放，输出三类对象：候选变更 Δ_i 、原始观测 O_i 和运行状态 σ_i 。执行面无权把三者自行解释为 PASS；即使命令返回 0，身份、日志完整性、oracle 和清理状态仍需交给证据面资格化。

单个节点只有一个对外 owner。实现者、验证者和只读审查者可以是节点图中的不同 owner，但不能同时争用同一工作区写权限或同一工程 shell。对于本地 RV64 RTL，机器合同进一步冻结 allowed_paths、write_paths、结构化 command/mode/purpose、必读上下文、交付物与成功条件。子 Agent 以隔离上下文执行；所需设计事实由合同显式提供，而非依赖父会话中未登记的叙事。

图 4 给出本地 RV64 节点从父图到证据提交的执行路径。只有合同创建、结构校验和渲染绑定均通过，owner 才得到执行权；若发现范围不足，当前节点返回 scope_extension_request，由控制面创建新版本合同并重新经过校验，旧合同不会被口头扩大。

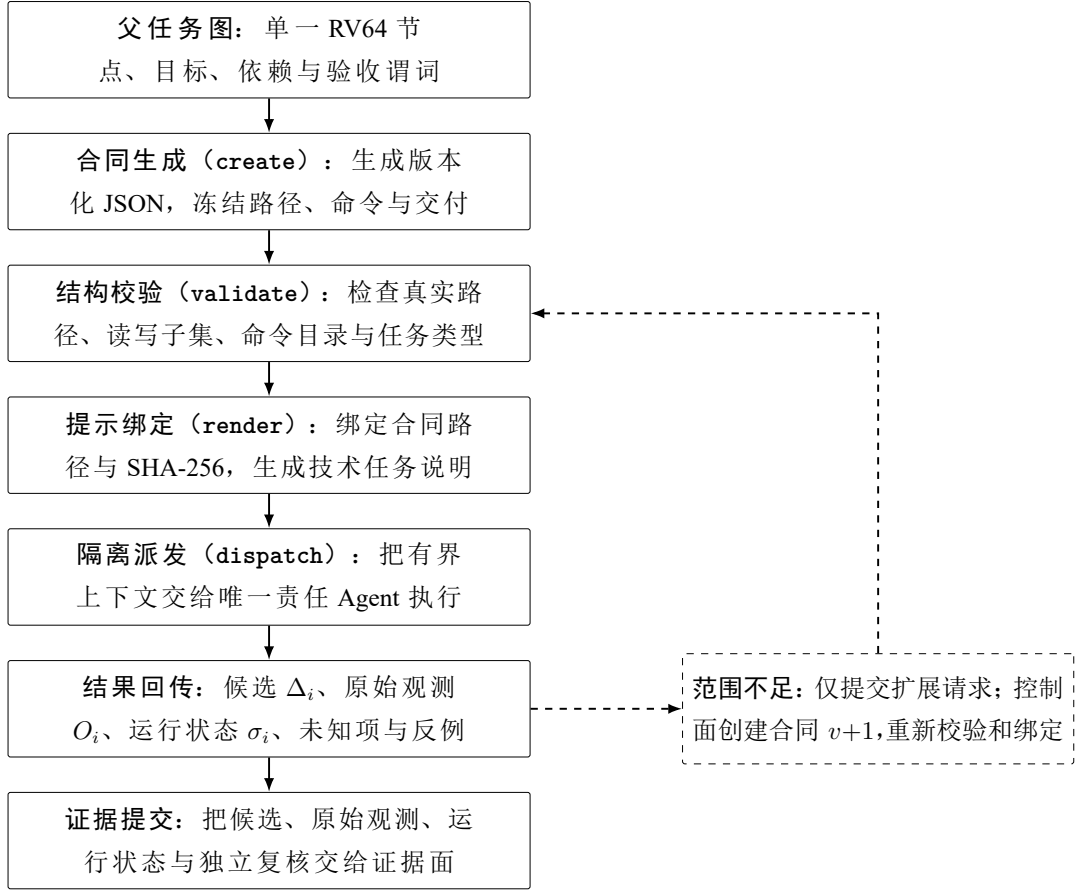


图 4: 本地 RV64 子任务合同的生成、校验、隔离执行与版本化扩展

上述流程的控制逻辑可压缩为代码 1。伪代码只描述分支与状态迁移；图 3 中的合同字段仍是节点定义的规范表达。

代码 1: 合同化 RV64 节点的最小调度算法

```

1 for node in topological_order(task_graph):
2     contract = create_versioned_contract(node, bounded_context)
3     require validate(contract)
4     prompt = render_with_path_and_sha256(contract)
5     result = dispatch(node.owner, prompt, inherit_parent_history=false)
6
7     if result.scope_extension_request:
8         replace_with_new_contract_version(node, result.request)
9         retry_from_validation(node)
10    else:
11        submit_to_evidence_plane(result.delta,
12                                result.raw_observation,
13                                result.run_state,
14                                result.review_report)

```

短命令与小时级作业共享同一合同接口，但状态保存不同。短命令可以直接回传返回码和 bounded log；长时间仿真、综合、STA 或系统回放还必须持续记录 stage、signal、evidence-complete 与 cleanup，防止交互终止或进程退出被误当作工程完成。这些状态仍只是执行事实，下一节才定义它们怎样成为可接受证据。

5.5 证据面：从原始观测到限定结论

证据面回答“哪些执行结果可以被相信，以及只能在什么范围内被相信”。它将执行面的 Δ_i, O_i, σ_i 与设计、配置、工具、workload、oracle 身份 I_i 绑定，再按验收谓词 Q_i 资格化：

$$\mathcal{A}_i = \text{qualify}(\Delta_i, O_i, \sigma_i, I_i, Q_i).$$

资格化至少检查对象一致性、原始日志完整性、预期终端 marker、正反例判定、返回码、信号与清理状态。独立审查报告可以成为 O_i 的一个输入，但审查文字不能覆盖 raw log，也不能把合同外对象纳入结论。

表 10: 证据面的状态及其可消费边界

状态	形成条件	下游允许的动作
PASS	本节点的身份、运行、证据、oracle 与清理全部满足合同	仅在该节点声明范围内作为硬依赖
FAIL	已观察到合同内反例、命令失败或中断状态	定位、回滚或生成下一实验；保留原始失败
GAP	输入、身份、观察点或覆盖不足，无法作出真假判定	请求补证或扩展合同；不得自动当作 PASS
UNQUALIFIED	作业完成，但量测或物理条件不足以支撑性能/PPA 结论	仅保留诊断价值，不允许晋级或 Pareto 比较
BLOCKED	上游合同、上下文、发布或基础设施前提未闭合	停止下游消费，先修复阻塞对象

对于长时间作业，最小 PASS 条件为

$$\text{PASS} = C_{\text{run}} \wedge C_{\text{evidence}} \wedge C_{\text{cleanup}} \wedge \neg C_{\text{signal}}.$$

所以“进程退出 0”只满足 C_{run} ，并不等于工程成功。证据不完整、清理失败或收到 HUP/INT/TERM 时，状态必须失败关闭。原始 FAIL 可以由修订后的 oracle 在冻结输入上产生一个新的 replay 结论，但旧状态不得被追溯改写。

本文最终把第 k 轮有效交付定义为

$$\mathcal{D}_k = \langle \Delta_k, \mathcal{E}_k, \mathcal{P}_k, \mathcal{V}_k \rangle,$$

其中 Δ_k 是工程变更, $\mathcal{E}_k$ 是正向与负向证据, $\mathcal{P}_k$ 是设计、构建、workload、oracle 与原始产物的来源绑定, $\mathcal{V}_k$ 是限定判定。任何一项缺失, 产物只能保持候选、GAP 或 UNQUALIFIED; 若缺失源于上游合同、上下文、发布或基础设施前提未闭合, 则状态为 BLOCKED。这些状态均不能成为有效交付。至此, 四个面的职责已经定义完整; 下一节只说明它们如何完成一次任务, 不再引入新的组成层。

5.6 四面协同的一次数据流

一次任务沿图 5 的实线主干前向运行。控制面首先产生节点合同 N_i ; 知识面依据合同构造有界上下文包 B_i ; 执行面产生候选与原始观测 $X_i = \langle \Delta_i, O_i, \sigma_i \rangle$; 证据面返回资格化对象 $\mathcal{A}_i$; 最后仍由控制面依据任务图决定接受、返工、保持 GAP 或终止。图中 C_{prepare} 与 C_{decide} 是同一控制面的两个生命周期端点, 不是两个控制面。

四类对象不能混在一条“成功日志”中: N_i 证明动作被授权, B_i 证明上下文有来源, X_i 记录实际执行, $\mathcal{A}_i$ 才给出可消费结论。任一对象变化都会使其后的对象失效。例如 design-id 或 workload 改变后, 旧 X_i 不能继续支持新的 $\mathcal{A}_i$; oracle 改变时可以对冻结 O_i 产生 versioned replay, 但不能重命名旧的 raw 状态。

5.7 外层能力晋级的操作阈值

图 1 已经给出外层反馈边; 要使“能力扩大”成为可证伪命题, 还必须规定一个环境改动经历多少次复现、多少次跨合同复用和多长时间稳定窗口后, 才允许从单次实现晋级为可复用能力。

对一次环境改动 z , 原定义只要求规则进入真源 $R(z)$ 、接入可发现能力 $W(z)$ 、经正例和针对性反例检验 $Q(z)$, 并被后续业务实际复用 $A(z)$ 。这仍不足以回答“复现几次、复用几次、稳定多久”。本文因此为后续审计预先冻结操作阈值 $\theta = (3, 2, 14\text{d})$: 同一版本能力在冻结输入和验收准则下至少有 3 次连续独立 replay; 至少被 2 个不同 task contract 调用, 且两个合同在模块、workload 或任务类别上至少一项不同; 从首次到末次合格调用跨越至少 14 天, 并保持同一 artifact hash, 或以显式新版本重新资格化。外层能力扩展被重新定义为

$$\text{Ext}_\theta(z) = R(z) \wedge W(z) \wedge Q(z) \wedge A(z) \wedge [n_{\text{replay}} \geq 3] \wedge [n_{\text{reuse}} \geq 2] \wedge [\Delta t \geq 14\text{d}].$$

这些数值不是从历史结果回算出的通过线, 也不用于追认现有构件。三次 replay 用于排除一次性偶合, 两个合同用于区分“单任务可用”和“跨任务复用”。14 天也不是由本项目平均开发间隔拟合而来, 而是预先选择的两个七日观察周期: 其目的只

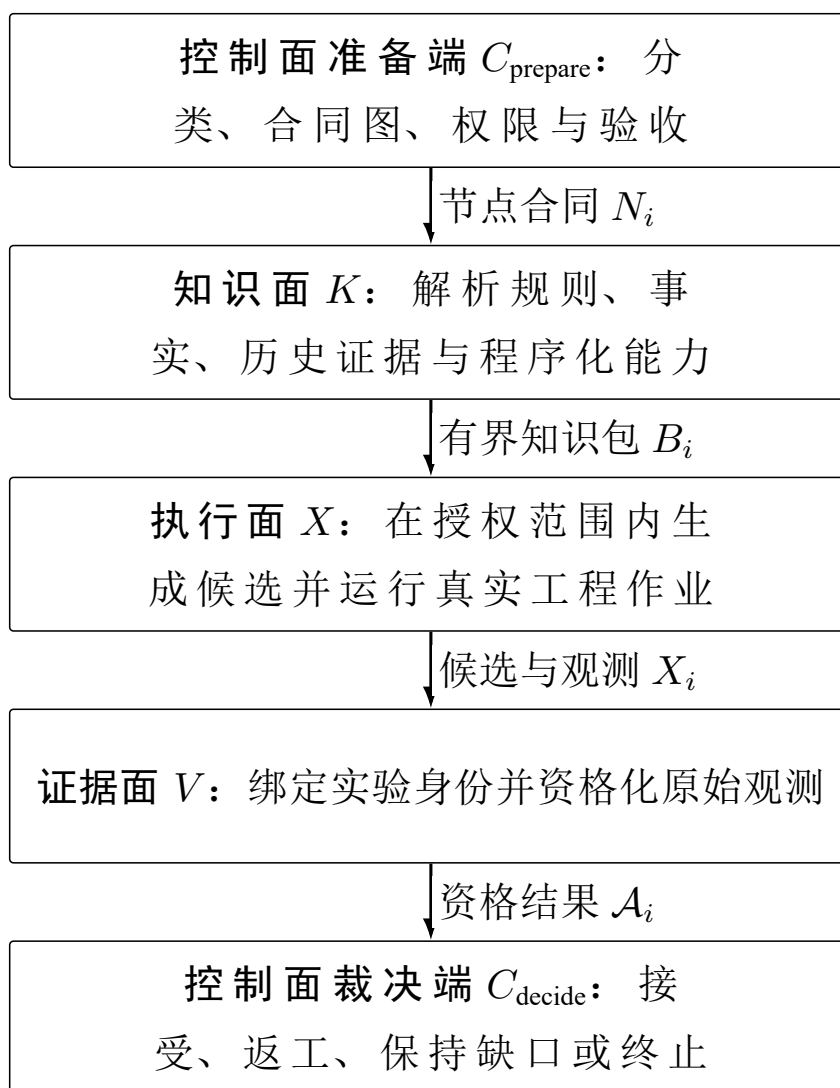


图 5: 一次任务对四面的调用顺序; 该图只表示前向数据流, 不表示反馈周期

是避免把同一短时调试会话内的重复调用误写成跨轮稳定性。因此，整组阈值只是兼顾审计成本与陈旧风险的工程启发式，不具有统计最优性，也不是跨项目定律。后续评价还应报告 **replay** 阈值 2/3/5、复用阈值 1/2/3 与稳定窗口 7/14/30 天的敏感性。未达任一项的对象只能称为候选能力或单次实现，不能进入有效能力集合。相应地，环境条件 Γ 下的有效 **Agent** 能力写为

$$C_{\text{eff}}(\Gamma; \theta) = \{\mathcal{T} \mid \mathcal{T} \text{ 能在固定验收标准下形成某个 } \mathcal{D}, \text{ 且新复用构件满足 } \text{Ext}_{\theta}\}.$$

只有验收标准不变且 $C_{\text{eff}}(\Gamma_1; \theta) \subsetneq C_{\text{eff}}(\Gamma_2; \theta)$ 时，才称环境能力扩大。当前主张映射尚未为每个历史构件完整编码 **replay** 次数、复用次数和稳定窗口，因此本文不把纵向里程碑解释为这一严格集合关系已被证明；它们只说明若干构件进入了后续任务。**Token**、代码行和脚本数量同样不能替代阈值审计。后文的双闭环运行机制将在不增加新架构元素的前提下，形式化两条反馈路径的状态、触发条件和停止规则。

6 四面的工程实例化：从方法接口到工作区载体

6.1 映射原则与边界

四平面方法不要求使用某一种语言或目录结构。本工作区的参考实现选择 **Markdown/JSON** 表达专业知识与机器合同，以 **C** 保存低开销任务生命周期，以 **shell** 连接工程工具，以 **Python** 实现合同校验、证据索引和原子发布。本章严格按照控制、知识、执行、证据的顺序展开；七张逐文件、逐字段追踪表统一移至附录 A，正文只保留决定数据流、责任边界和判定范围的实现机制。表 22 给出完整职责映射。实现载体可以跨面：例如 **E2E** 的定义属于知识面，选择属于控制面，运行属于执行面，结果资格化属于证据面；因此不能用“某个文件属于哪个目录”代替接口分析。

6.2 控制面的实现：生命周期、代次与门禁路由

控制面位于目标合同与真实工程节点之间。它不负责实现 **RTL**，也不替代业务验证，而负责回答：当前工作属于哪类任务、哪些路径由本轮拥有、哪些证据已经形成、何时需要独立复核，以及哪组固定门禁能够授权终止。其上游是用户目标、分类策略和显式作用域；下游是图执行器、**domain gate**、证据面与归档器。没有这一层，**Agent** 可以在多轮对话中继续修改，却无法机械区分“同一候选的复核”与“已经产生新候选”。

参考实现采用一个轻量生命周期控制器。`scripts/agent-flow.sh` 只是可移植启动适配器：仅当 **C** 源变化或缓存二进制不存在时，使用 `C11 -O2 -Wall -Wextra -Werror` 编译临时文件并原子替换；真正的状态与门禁路由位于 `scripts/agent-flow.c`。这种 **C+shell** 分工不是论文贡献本身，而是为日常短任务降低解释器和全

量 profile 开销的实现选择。FlowMeta 保存 task、class、archive mode、status、耗时和 generation；后者承担旧证据失效语义：每次登记或再次触碰路径/门禁，代次递增且状态回到 ACTIVE，只有 gate ID、状态和 generation 同时匹配的旧门禁结果才允许复用。由此，新候选不能消费旧代次 PASS。承重实现节选集中于附录代码 11，正文只保留该控制不变量。

生命周期操作的抽象语义如下。命令名只是参考接口：

固定 gate registry 不接受 Agent 临时拼接任意 shell 字符串；它把显式路径映射到已注册的验证入口。因此，控制面只选择“哪一个既有判定器承担本次责任”，不能为获得绿色结果临时改写判定命令。任务生成新路径、重新分类或补登记门禁时 generation 递增，旧候选缓存立即失效；只有同一代次的候选、审查和正式结束可以复用门禁结果。这样，“完成”由 class policy、作用域、真实 evidence 和固定 checker 共同决定，而不是由模型自由解释。

附录代码 12 给出这一映射的三个真实分支。修改端到端配置时选择“执行剖面接线检查”，修改任一 Skill 时选择“Skill 审计”，修改 RV64 子任务合同的规则、生成器或策略时还要增加“RTL 合同检查”。因此，路径不是事后报告中的装饰信息，而是控制面选择验证责任的输入；固定门禁也不会因为 Agent 更换措辞而变化。

6.3 知识面的实现：专业知识如何编译为可执行能力

知识面的关键不是存放更多 Markdown，而是把不同语义的知识放入不同载体，并沿固定编译链进入执行。instruction 表达“什么不能被违反”，Skill 表达“怎样稳定地产生产物”，machine contract 固定“本次允许读写与运行什么”，profile/E2E 表达“这些能力怎样组合才能证明完成”。它们的关系为

$$\begin{aligned} \text{professional invariant} &\rightarrow \text{executable procedure} \rightarrow \text{task contract} \\ &\rightarrow \text{graph node} \rightarrow \text{domain evidence.} \end{aligned}$$

例如，接口策略要求在修改 RTL 前冻结握手、stall、flush/redirect、异常序、访存序和投机恢复；RTL 生成策略又要求先明确需求、状态机、不变量、数据通路与拓扑。任务 Skill 随后把这些义务参数化为 allowed_paths、write_paths、结构化 command/mode/purpose、success criteria 和反例出口。若发现范围不足，子节点只能请求新版本合同，不能通过一句补充说明改变旧边界。profile 再把合同绑定到 owner、函数、输入输出和依赖闭包，最后由真实 testbench、DiffTest、系统或 EDA 节点承担业务结论。

6.3.1 从接口经验到机器合同

知识编译链可以由一条真实 RV64 规则说明。任务合同 **instruction** 规定，任何子任务都必须显式给出表 25 中的字段；缺项不得派发。这些字段不是为了增加文档篇幅，而是分别封闭一个常见失效源：目标含混、源码范围无限、只读审查意外写入、自由拼接命令、缺少必要上下游，以及“工具返回 0”被直接解释为技术完成。

Skill 随后把这些声明式字段编译为实际合同。附录代码 13 节选自合同生成器：允许路径由用户范围、必读材料和合同自身组成；命令的模式与用途来自固定目录；交付和成功条件原样进入 JSON。因此，**Skill** 的产物不是另一段 **Prompt**，而是控制面和审查者共同消费的机器对象。

合同校验不是格式检查。生成器会拒绝“写路径不属于可读路径”“只读审查声明写入”“写型命令没有写路径”以及“无工具模式仍携带命令”等组合。由此得到的边界仍需进入 E2E 才能成为可发现能力：执行剖面的一行同时绑定节点名、实现模块、节点函数、责任主体、五类输入和预期输出；节点函数再实际运行生成器的审计、自测与命令行自测。表 26 给出该接线，而不是只说明文件存在。

这个例子给出三层不同的可信性：**instruction** 证明规则被明确表达，**Skill** 证明规则可以生成受限任务，E2E 证明能力接线可运行；只有最后的业务验证才证明合同内的 RV64 技术主张。把三者合并成一句“E2E 完成知识”，会同时夸大环境检查的结论并掩盖真正的业务 **oracle**。

6.4 执行面的实现：确定环境、图闭包与节点作业

执行面的参考实现把知识面选出的 **profile** 展开为有序节点，并为每个节点建立确定的工具环境。其物理主链到“产生原始观测”为止；**report**、**manifest** 和 **publication** 属于下一节的证据面，不能因为它们由同一 **runner** 调用就混成一个职责。

代码 2: **profile** 从任务身份到原始节点观测的执行主链

```
1 task_slug + selected profile
2   -> rebuild active live index
3   -> brief(non-history focus, bounded budget)
4   -> recursively resolve @include profile closure
5   -> bind node/module/function/owner/inputs/outputs
6   -> load deterministic tool environment
7   -> dispatch node function
8   -> raw evidence/<node_id>.log + rc + runtime state
```

图执行器只承担通用编排，不把全部工程语义塞入一个 **shell**。**profile** 使用 **node/module/function/owner/inputs/outputs** 六元组；**@include** 递归展开可复用子图；**module library** 实现具体节点函数。**scripts/agent-env.sh** 在所有非交互节点

之前固定仓库根、NEMU、AM、NPC、Yosys/STA、RISC-V 工具链与可选 JDK 路径, `scripts/agent-run.sh` 将同一环境提供给手工或单节点命令。因此, `profile` 的上游是机器合同与能力索引, 下游是具有确定环境的 `domain command`, 而不是另一个自然语言规划器。

6.4.1 工程执行后端及其原始产物

NEMU、NPC、AM和 EDA 工具不是论文结论, 也不是同一种“仿真器”。它们位于执行链的不同位置, 分别产生参考端观测、目标端观测、工作负载镜像或实现分析报告。为避免不熟悉本工作区的审稿人把工具名称误当作验收结果, 表 27 给出各组件的工程含义、直接产物与证据边界。

这组后端形成两条可区分的数据流: 程序源码经交叉工具链与 AM 形成冻结工作负载, 随后分别进入 NEMU 参考端和 NPC 目标端; DiffTest (差分测试) 逐条比较两端退休后的架构状态, 系统级判定基准则检查更长事务的终止条件。RTL 另经 Yosys 形成网表, 再由 OpenSTA 消费网表与约束形成时序观测。前一条回答功能状态是否一致, 后一条回答结构与时序代理量如何变化, 二者不能互相替代。

环境自身的 `agent-system profile` 是该模式的一个实例。九个物理配置项通过一个 `discovery include` 展开为十一节点, 但论文不把九行文件当作方法; 表 28 按方法责任把节点重新分组。

证据与边界: 环境自身的运行证据。一次已发布环境自验证中, `profile` 在 42 s 内展开 11 个节点, `manifest` 记录 PASS=11/11 和 13 个 `ordinary evidence payload`; `manifest` 入库后, `evidence API` 共可查询 14 个资产。`completion-publication.md` 绑定 `task report`、`manifest`、`context brief`、`profile resolve`、`evidence index`、`dispatch log` 与 `nodes.tsv` 七份 SHA-256。更能说明失败关闭的是另一次故障注入: 十一业务节点均为 PASS, 但 `context brief` 因独立 `focus` 不足产生 WARN, `overall` 仍被发布为 `blocked`; 随后使用具有工程语义的 `OooIntBackend task slug` 重跑才形成 `completed publication`。这里被拒绝的不是 RTL, 而是一条缺少可信启动上下文的环境完成声明。

这一 `profile` 验证的是 AI 环境自身是否可发现、可执行和可追溯; 它不能证明 NPC、NEMU、Linux 或 PPA 正确。业务目标必须使用自己的 `testbench`、`DiffTest`、系统或 EDA `evidence`。这种边界是方法的一部分: 环境 GREEN 与设计 GREEN 不能互相替代。

6.5 证据面的实现: 资格化、失败关闭与原子发布

6.5.1 长跑终止语义适配器

长时间仿真、综合、STA 或系统回放的进程状态只属于执行面; 工程 PASS 属于证据面。Loop-Work 因而规定一个最小提交条件:

$$\text{PASS} = C_{\text{run}} \wedge C_{\text{evidence}} \wedge C_{\text{cleanup}} \wedge \neg C_{\text{signal}},$$

其中 C_{run} 表示主命令正常结束, C_{evidence} 表示规定 marker、哈希和检查已到末端, C_{cleanup} 表示配置恢复与临时资源清理成功, C_{signal} 表示收到 HUP、INT 或 TERM。进程返回 0 只是第一个条件, 不能单独提交工程状态。

参考实现 `scripts/task-run-status.sh` 是执行面与证据面的终止语义适配器: runner 在启动时写 RUNNING, 随工程推进更新 stage, 只有全部证据检查结束后才能置 evidence-complete, 最后再提交状态。test-task-run-status.sh 对附录表 29 的八类路径逐项断言; 因此这段 shell 的意义不在于四个 if 条件本身, 而在于它为所有小时级 runner 提供了统一、可故障注入的提交协议。该协议正是十九小时 Ubuntu 事务、综合和 STA 能够在中断后保持可审计边界的公共下游接口。

6.5.2 证据原子发布、有界召回与方法自验证

证据提交以后仍不能直接进入长期知识。发布器把节点标识、来源执行剖面、实现模块、责任主体、函数、状态、输入、输出和证据路径组成节点元组, 再与原始观测及运行身份绑定; 对应机器字段为 node/source-profile/module/owner/function/status/inputs/outputs/evidence。系统依次生成节点表、调度日志、任务报告、清单和证据索引; 随后重算普通证据的路径、尺寸与 SHA-256, 验证执行剖面闭包和事件顺序, 最后才生成完成标记。完成态先同步暂存文档, 再在单个数据库事务中复核完成标记、发布记录与产物集合; 任何净化、索引、哈希或数据库步骤失败都会撤销本轮发布。参考实现主要位于 `scripts/e2e/lib/report.sh`, 但其抽象职责是“证据原子提交”, 不是格式化报告。

下一轮只从已发布对象中选择最小充分历史。当前有效的 instruction、Skill 和执行剖面保持在线; 稳定 memory 与任务档案可查询; 原始日志只向上下文提供路径、尺寸、SHA-256、标记和有界摘要。启动摘要受执行剖面、非历史主题和 2400-token 硬预算约束; 历史运行通过 run-id 与证据索引按需读取。这样, 证据面可以向知识面反馈已验证结论, 而不会把全部聊天和重型日志再次灌入模型。

方法自身同样接受分层验证。轻量模式只检查控制器与 shell 状态协议; 目标级模式追加 schema、policy、artifact、Skill、合同和 profile 绑定审计; release 模式再构建相对路径包、扫描本机路径与私有 marker, 并复核数据库与交付边界。参考实现由 `agent-maintain.sh` 编排, 并由 `package-ai-dev-env.sh` 形成可迁移包。两者位于外层能力闭环的末端: 只有规则、执行入口、反例 gate 和交付包同时成立, 一项环境修改才算从个人经验变成可复用工程能力。

6.6 一个复合合同的端到端映射

前四节分别说明了四个面的实现，本节只用一个真实复合合同把它们串起来，不再引入新的组件。该合同的根目标是复核当前 `PERF_BASELINE`：核算 lane1 到 lane0 的计数端点修正，验证 v4 cycle/slot 守恒、三次 bit-exact 结果与 stats-off 等价，确认 A1（第一次原始性能基线执行）和 A2（同一冻结配置下的第二次独立执行）的 FAIL 未被覆盖，并在缺少物理实现条件时保持 PPA 为 UNQUALIFIED。这里的 reviewer 是前文已定义的独立只读审查节点；它只复核冻结输入，不修改 RTL，也不代替 benchmark、综合或 STA。

该复合主张沿“根目标归一化 → 知识装配 → 合同编译 → 隔离复核 → 资格判定 → 节点终止”的连续数据链传递；每一步都以前一步形成的对象为输入。完整逐步映射移至附录表 30。

其中，合同的机器边界可概括为附录代码 14。实际 JSON 还保存完整路径、合同版本和 SHA-256；此处只保留决定权限与结论范围的字段。

合同生成器的结构校验为 PASS，只说明路径、命令、任务类型和回复出口合法；它不证明性能技术结论。为检验这条边界，Skill 自测还构造路径穿越、只读任务写入、错误 purpose、no-tools 混入命令以及固定“最多发现 N 个问题”等非法合同，验证器必须逐项拒绝。这个实例展示的是同一复合主张如何沿四面传递，而不是把某个 JSON 或 Python 文件本身包装成论文方法。

7 四面协同的双闭环运行机制

顶层方法已经定义四类责任状态和两条返回路径，前一章给出了同构参考实现。本章只在时间轴上形式化这些既有接口：内层闭环优化当前任务，外层闭环把重复且可迁移的缺口固化到控制、知识、执行或证据面。双闭环不是新的架构层，也不改变四面的职责边界。

7.1 状态、事件与反馈对象

本文把第 k 个工作轮的状态写为

$$x_k = (D_k, C_k, V_k, L_k, P_k),$$

其中 D_k 为设计快照， C_k 为配置与输入身份， V_k 为验证器/oracle 版本， L_k 为未闭合义务账本， P_k 为当前成熟度。这只是状态记录的类型定义，并不是预测模型。本轮合同冻结有限验收谓词集合 $Q_k = \{q_1, \dots, q_n\}$ ，并为每个谓词给出期望状态 $\hat{o}_k(q)$ 。真实工具产生的观测状态限定为

$$o_k(q) \in \{\text{PASS}, \text{FAIL}, \text{GAP}, \text{UNQUALIFIED}\}.$$

身份不一致、证据不完整和 freshness 失效形成 blocker 集 B_k ；本轮未闭合项被操作性地定义为

$$E_k = \{q \in Q_k \mid o_k(q) \neq \hat{o}_k(q)\} \cup B_k.$$

BLOCKED 不是工具观测值，而是 $B_k \neq \emptyset$ 时控制面对下游消费作出的阻塞裁决。这里没有抽象的“差值”或待学习的反馈函数： o_k 必须来自编译、仿真、testbench、综合/STA、退出状态、计数器或身份哈希； E_k 的每一项必须能回指原始观测。动作选择遵循固定优先级。若 $B_k \neq \emptyset$ ，保持 GAP 且不猜测性修改 RTL；若候选触发 hard gate，先回滚并复验；若确认是 oracle 缺陷且原始日志完整，只生成版本化 replay；其余情况从能够区分现有竞争假设的实验中，按 focused、module、system、PPA 的成本层级选择最低层实验，同层并列时采用合同中预先登记的顺序。 x_{k+1} 是动作执行后由收据记录的新状态，而不是公式预测值。由此，反馈规则可以由另一位工程师按相同合同复核，无法判别时则保持 GAP。

7.2 内层交付闭环：失败不是统一重试

同一条 FAIL 对应的正确动作可能完全不同。表 11 给出工作区已实际使用的分流。

7.3 内层算法与停止条件

算法强调三个不等式：

$$\text{SCOPED_ACCEPTED} \neq \text{PROMOTED},$$

$$\text{replay PASS} \neq \text{raw live-run PASS},$$

$$\text{benchmark completed} \neq \text{measurement qualified}.$$

7.4 外层能力闭环：交付单元与跨轮知识固化

有效产物不是“Agent 的回答”，而是可重建状态迁移的最小证据包。

最后一行使 Loop-Work 同时成为环境更新循环。不是所有反馈都应写入同一处：本轮 design-id、风险和已知事实进入 retained memory；一次运行与原始失败进入 task-run/evidence index；可复用的人类决策规则进入 instruction，短小且需要重复操作的流程才封装为 Skill；能机械裁决的字段进入 JSON schema、checker 或 assertion；需要自动发现和执行的能力再接入 profile 与 e2e function。其数据流可写为

表 11: Loop-Work 的反馈分类与下一动作

观测类型	判别问题	正确动作
DUT 功能或周期失败	是否定位到首个错误事务及 现有合同违反	最小 RTL 修复, 新 design-id, focused 后逐层回归
身份/freshness 失败	当前结果是否属于不同 RTL、 filelist、配置或 workload	重建或重跑证据; 不修改 RTL 猜测原 因
oracle 假阳性/假阴性	DUT 是否到达终态, 原始日 志是否完整	修正 oracle, 做正/反例测试, 对冻结 日志版本化重放
mutation 未触发/未传 播	负向变体是否真的改变并到 达观察点	保持 GAP, 修复刺激或 observation, 不宣称敏感
量测不合格	counter 是否守恒、身份是否 稳定、观测是否非干扰	拒绝性能候选, 先修量测合同或探针
signal/证据/清理不完整	是否到达预定证据末端	保留 FAIL 与 stage/signal, 禁止自动 PASS
hard gate 回归	候选是否破坏上级合同或 guardrail	回滚到最近合格基线, 并复验恢复

表 12: 每个 Loop-Work 的最低产物

产物	内容	下游用途
task/contract	分类、目标、允许路径/命令、 成功条件	约束 Agent 动作和审查范围
input/identity manifest	RTL、配置、工具、workload、 oracle 的路径与哈希	判断证据能否比较或复用
raw observation	log、trace、counter、wave、 exit code、signal	保留未被摘要改写的事实
verification receipt	正向、负向、marker、计数与 边界	判断 oracle 与门禁是否合格
decision trace	hypothesis、竞争假设、反例、 回滚与选择理由	使下一轮不再盲目重复
status/publication	PASS、FAIL、GAP、 UNQUALIFIED 及 stage/cleanup/review	限定可消费的完成主张
reusable update	稳定结论进入 memory/skill/instruction/checker	改善下一轮环境而非堆积临时日志

代码 3: Loop-Work 的最小执行算法

```
1 x <- bind(design, config, inputs, oracle, maturity)
2
3 while budget_available:
4     r <- freeze_scoped_claim_and_gates(x)
5     open <- classify_unmet_gates_and_blockers(x, r)
6     a <- select_by_gate_priority_then_cost(open, contract_order)
7     y <- execute_and_observe(x, a)
8     A <- seal(identity, commands, rc, raw_output, y)
9
10    if identity_mismatch(A) or evidence_incomplete(A):
11        return GAP(A, "binding/evidence")
12
13    e <- compare(r, y)
14    if violates_hard_gate(e):
15        rollback_to_last_qualified_state()
16        verify_rollback()
17        record(REJECTED_OR_ROLLED_BACK, A, e)
18        continue_or_stop()
19
20    if scoped_error_is_zero(e):
21        if promotion_requested and parent_gates_pass_same_identity(A):
22            return PROMOTED(A)
23        return SCOPED_ACCEPTED(A, promotion=false)
24
25    if error_is_actionable(e):
26        x <- transition(x, a, e)
27        continue
28    return GAP(A, e)
```


engineering counterexample → stable rule extraction

→ instruction/Skill/checker → profile wiring

→ qualified execution → next engineering action.

这种分流避免两种相反错误：其一，把所有历史过程塞进 memory，使下一轮被过时细节淹没；其二，只在某次最终回复里记住规则，导致换会话或换模型后能力消失。若一条稳定规则已经写入 instruction 但没有 profile/checker 能发现，环境仍处于“知道但做不到”；若 profile 能执行但没有 mutation 或反例证明其会拒绝错误输入，则处于“能跑但不会判”。只有闭合这两个缺口，工程经验才形成 Agent 可执行、可检验的外部能力。

失败、回滚和未晋级也是有效产物。它们排除了一个候选方向，保护了基线，并为下一轮提供更窄的搜索空间。只有“既没有改变判断、也没有留下可验证状态”的循环才是无效消耗。

附录表 31 给出四条“工程反例—方法增量—后续复用”的过程链。它们说明新结构确实进入后续任务，但由于模型、代码和任务难度同时变化，只能作为外层闭环的过程证据，不能作为独立因果效应估计。

7.5 内外闭环的触发关系

不是每一次 FAIL 都应修改 AI 环境。单次 RTL 实现错误通常只在内层修复；只有缺口具有重复性、可迁移性，且无法由当前合同中的局部动作消除时，才进入外层。操作规则是：单次实现错误只修当前候选；跨节点重复且可抽象为稳定不变量的错误进入 instruction/Skill；公共 runner、oracle、环境或历史污染类缺口分别进入执行/证据、知识/E2E、环境适配或有界召回接口。完整分流见附录表 32。

外层修改完成后不能追溯改变旧 task-run。它必须形成新版本规则或执行器，先通过自身正例和针对性反例，再由后续业务任务实际复用；否则只是一项环境候选，而不是能力扩展。这样，图 1 上方返回通道的起点和终点都有可检验条件。

7.6 CPI/PPA 优化的受约束晋级链

处理器性能优化不是“修改 RTL 后观察 CPI 是否下降”的单步任务。CPI（每条退休指令的周期数）可能因测量窗口、退休端点、统计逻辑或 workload 身份改变而下降；PPA（性能、功耗、面积）也可能在功能错误、时序未约束或 memory macro 缺失时得到更好看的代理数字。Loop-Work 因而把一次优化定义为完整设计点沿表 13 逐节点晋级。节点之间是硬依赖：前一节点没有形成可消费收据，后一节点只能保持缺口，不能由 Agent 依据“趋势合理”跳过。

表 13: RV64 双发射乱序核的 CPI/PPA 晋级节点及拒绝边界

节点	必需输入	形成的产物	不能晋级的条件
架构发现 ARCH_DISCOVERY	当前 RTL、接口、配置、已有缺陷与验证入口	模块/事务图、债务清单、待闭合 owner 与观察点	调用链未知、设计身份未冻结
架构闭合 ARCH_CLOSED	逐项债务、接口不变量和定向验证	每项债务的修复、反例或明确 GAP	只通过局部正例，没有跨模块事务解释
历史缺陷下沉与架构门	已知 Linux/DiffTest/全核反例	最低充分层的 TB、断言或可编译负向 RTL，以及全量回归	历史错误只能在昂贵系统层复现，或判定器无法检出变体
架构稳定 ARCH_STABLE	同一设计快照上的功能、乱序、恢复和副作用证据	带 design-id 的稳定架构收据	production RTL、配置或验证身份仍在变化
性能量测合同	架构稳定收据、workload、ROI 与计数器定义	起止端点、重复次数、守恒式、非干扰检查和原始状态协议	ROI 或 lane 端点含混，统计开关改变 cycles/retired
性能基线 PERF_BASELINE	同一合同下的重复执行与独立核算	可重放 cycles、retired、CPI/IPC、计数器栈及限定收据	运行不重复、端点不守恒、原始 FAIL 被覆盖
瓶颈普查 (bottleneck census)	合格基线与计数器所有权定义	主瓶颈、竞争瓶颈、可证伪预测和最低成本区分实验	嵌套驻留被相加，或探针事件缺少事务身份
单机制候选与架构再门禁	一个瓶颈假设、一个可归因机制和回滚点	candidate design-id、A/B 原始日志、差分计数器与回归	同时改变多个机制，或优化改变周期/资源合同 后未重走架构门
CPI/PPA 联合资格	同源功能结果、netlist、约束、库、角、活动率和 macro	逐 workload floor、timing/area/power 资格与 Pareto 位置	任一功能硬门失败，或时序/功耗/面积物理条件不完整
系统再认证与可重复晋级	合格候选、系统 workload 和独立审查	系统收据、跨次重复结果及 PROMOTABLE 判定	仅局部或代理结果变好，系统和独立复核未闭合

7.6.1 架构发现、闭合与稳定

架构发现不是泛读整个仓库，而是围绕准备优化的事务建立可验证的生产者—消费者图。例如 memory stall 的 CPI 候选必须先回答：请求由哪个模块建立，reservation、bridge、AXI response 和 commit 各由哪个 owner 持有，flush 或 replay 能否清除它，以及哪个计数器能观察到每个生命周期。输出是带路径和未知项的债务表，不是 RTL 候选。

架构闭合逐项处理这些债务，并把历史系统缺陷下沉为低成本检查。若一项 Linux 故障可以由模块 TB 或断言稳定复现，后续 Agent 就不必用完整 Ubuntu 回放筛掉同类候选；若只能在系统组合中出现，则保留在系统层而不伪造低层覆盖。架构门要求功能回归、双发射/乱序合同、精确恢复、异常与副作用同时闭合，并用能够通过编译的负向 RTL 检查判定器是否真的对目标错误敏感。只有 production RTL 与配置冻结后，才形成 ARCH_STABLE；它表示性能测量的架构前提稳定，不表示 PPA 已经合格。

7.6.2 量测合同与性能基线

正式测量首先冻结 workload 二进制、输入、编译器、起止 PC/marker、cache/TLB 初态、memory latency、seed、计数器版本、重复次数和阈值。基本量为

$$\text{CPI} = \frac{\text{测量窗口内的核心周期数}}{\text{测量窗口内退休的架构指令数}}, \quad \text{IPC} = \frac{1}{\text{CPI}}.$$

双发射核每周期有两个退休槽。令 C 为冻结 ROI 的核心周期数， $R > 0$ 为退休指令数， $l_s, l_e \in \{0, 1\}$ 分别为日志裁剪起止端点的 lane 编号。若裁剪协议把可见槽跨度定义为

$$S = 2C + l_e - l_s,$$

且 $U_{\text{span}} = S - R$ 为该裁剪跨度内的未使用槽，则由 $2C = R + U_{\text{span}} + l_s - l_e$ 可得

$$\text{CPI} = \frac{C}{R} = \frac{1}{2} + \frac{U_{\text{span}} + l_s - l_e}{2R}.$$

因此，原先不含端点项的写法只有在 $l_s = l_e$ ，或把 $U_{\text{corr}} = U_{\text{span}} + l_s - l_e = 2C - R$ 明确定义为“端点修正后的未使用槽”时才成立：

$$\text{CPI} = \frac{1}{2} + \frac{U_{\text{corr}}}{2R}.$$

为避免 lane 被误解为半个物理周期，新的量测合同优先使用固定半开周期窗口 $[c_0, c_1)$ ，令 $C = c_1 - c_0$ 、 $U_{\text{full}} = 2C - R$ ；此时上式直接成立且不再需要端点 lane。旧 A1/A2 日志仍按其原裁剪协议重放，但收据必须显式保存 $l_s, l_e, U_{\text{span}}$ 与 U_{corr} ，不能混用。

本工程中的 A1 是第一次原始性能基线执行，A2 是同一冻结配置下的第二次独立执行。二者都被旧判定器因 Dhrystone 的 ROI 从 lane1 结束、在 lane0 重新开始而拒绝；原始状态至今保持 FAIL。后续版本化勘误只把错误的“起止 lane 必须相等”代理条件替换为上述精确容量式，并对冻结 A2 日志重放；设计、workload、ROI、原始日志和旧状态均未改变。CoreMark 与 Dhrystone 各三次统计开启运行 bit-exact，统计

关闭运行的 cycles/retired 又与开启时相同，才形成 PERF_BASELINE。这组证据证明量测接口可重复且统计逻辑没有观察到干扰，不证明 CPI 已经因某次 RTL 修改而改善。

7.6.3 瓶颈普查、单机制候选与反馈

性能基线形成后，Agent 才能建立 lost-slot census（退休槽损失普查）。总 cycles、unused retire slots、I/D-cache miss、控制等待、LSU 等待、AXI 等待和事务 owner 驻留分别记录；这些计数可能在同一周期重叠，不能简单相加为总周期。每个主瓶颈必须附带竞争解释，并预测“若主假设成立，哪个 workload、哪个 counter、哪个事务阶段会怎样变化”。控制面优先调度能区分两种解释的最低成本实验，而不是直接要求模型改 RTL。

一个候选只允许包含一个可归因机制，并按同一 design/build/workload 身份执行交错 A/B，例如 A-B-B-A-A-B；每次原始日志独立保存，比较逐 workload 的中位数、最差值和守恒项。候选若改变周期、资源数量、调度、恢复或事务生命周期，就被重新分类为架构变更并返回架构门。本工程中，放宽双 memory 发射使 CPI 从 0.645 退化为 0.665，候选被回滚；另一次只读 owner 探针虽然完成六次 benchmark，但事件身份全部无效，因此控制面没有生成 production RTL candidate。这两个结果分别证明负优化和无效诊断不会因“程序跑完”而进入下一节点；它们不证明所有未来瓶颈均能由现有计数器识别。

7.6.4 PPA 资格、系统再认证与可交付边界

联合资格按固定全序裁决：同源设计与完成定义、功能、双发射/乱序、精确恢复与副作用、精确 timing tier、逐 workload floor、面积/功耗资格、Pareto 前沿，最后才执行晋级检查。所谓 Pareto 前沿，是不存在另一个设计点在 CPI、频率、功耗和面积上均不差且至少一项更好的候选集合。平均 CPI、generic cell 数或一个综合评分不能抵消前面的硬门。

物理资格至少要求同源 netlist、SDC、library/corner、完整 I/O delay、真实 memory macro、有效切换活动、时钟树/寄生与对应 timing 报告。当前 Yosys/OpenSTA 结果能够用于结构筛选，却因 I/O 未约束、缺少 SPEF、CTS、OCV、真实宏和功耗活动而保持 UNQUALIFIED。这种非晋级状态不是“尚未整理好的成功”，而是一个可消费的工程结论：允许继续定位面积或逻辑深度，不允许宣称频率、面积、功耗签核，也不允许进入系统再认证和可重复 promotion。

表 14: 现有证据对 CPI/PPA 工作流的支持范围

已观察事件	若无分层门禁可能得到的错误结论	该工作流实际形成的结论及其意义
A1/A2 旧判定器拒绝、冻结重放通过	删除 FAIL，只保留新的绿色基线	原始 FAIL 与新收据并存；证明判定器版本可修订但历史事实不可改写
三次重复与统计开关非干扰	一次 CPI 数字即可比较候选	只授予可重复量测基线；为后续同身份 A/B 提供共同分母
六次 benchmark 完成但 owner 事件无效	workload 完成即证明瓶颈归因成立	在收据构造阶段失败，阻止错误归因进入 RTL 搜索空间
双 memory CPI 退化	局部回归通过即可保留“优化”	候选回滚并保留负实验；后续 Agent 不再重复同一搜索方向
代理综合和时序数字可读	直接写成目标频率、面积或功耗改善	保持 PPA 未资格化；把缺失物理条件转化为后续输入而非夸大结论

因此，本节证据支持的是“工作流能把量测错误、负优化、无效归因和物理条件不足导向不同终态”，并形成下一实验所需的输入；它尚不支持“该方法已经带来稳定 CPI 提升”或“当前核已经完成物理 PPA 签核”。这种区分使性能工程的有效产物不仅包括更优设计，也包括被可靠淘汰的候选和不能晋级的边界。

8 案例研究设计与证据协议

前述章节给出了方法及其实现接口；在进入工程里程碑之前，本章先固定“哪些材料可以作为证据、怎样选择和编码主张、冲突怎样处理”。这样，后文的时间顺序只用于过程追踪，而不会被直接解释为方法产生效果的因果顺序。本文采用单仓库纵向嵌入式案例研究，并借鉴软件工程案例研究和过程追踪对案例边界、分析单位与诊断性证据的要求 [33, 34]。

8.1 案例边界与分析单位

案例是持续演化的 RISC-V 处理器工作区；分析单位不是目录、文件或一次聊天，而是任务 episode 中可独立判真的工程主张。一个 episode 具有明确目标、待判断对象、可恢复验收条件，以及 accept、reject、rollback、GAP、UNQUALIFIED、interrupted 或 MISSING 中至少一个终态。观察窗为 2026-04-01 至 2026-08-04，数据冻结提交为 858fd3cf0dc6cb466c80707051f40d4a4710fcbc。版本化源码、合同、task-run 状态、原始日志、结构化收据和证据哈希进入分析；只存在于未归档对话中的推测不进入证据集。

8.2 主张选择、编码与冲突处理

30 条目的性主张由任务终态、里程碑和阶段性报告抽取，并统一规范为“工程对象—实验身份—验收谓词—结论—边界”五元组。重复发布合并为一个 `episode`，强弱不同或互相矛盾的主张分别保留，再按 RQ1–RQ4、验证压力、正负结果与证据状态作最大差异选择。纳入项必须可被反驳、有精确来源路径、能恢复至少两类身份/执行信息，并具有明确的接受、拒绝、回滚或未资格边界；纯进度文字、配置不可比的数字和无法恢复决策边界的临时日志被排除。该集合用于审计本文实际使用的论断，不作为任务总体的概率样本。

机器可读 `codebook` 把字段分为五组：主张与分类，`episode` 与身份，来源与执行，结果与反例，以及外推边界。证据状态只允许 `SUPPORTED`、`PARTIALLY_SUPPORTED`、`OBSERVATIONAL_ONLY`、`CONTRADICTED` 和 `MISSING`。只有在 `design`、`configuration`、`workload` 与 `oracle` 适用域相同而结论不同时，才构成证据冲突；新判定器可以生成版本化 `replay`，但保留原始状态。

第二 Agent 对预先固定的 12/30 主张执行隔离、只读的反例复核，检查状态、原始路径、数字、返回码、回滚和声明边界。复核者能够看到主编码，因而这是独立反例审查而非盲法双编码；三条初始主张因环境描述过强、身份冲突或原始路径不足被阻断并修订。样本规模和非盲条件决定本文不报告人类编码一致性指标 [35]。

8.3 数据保留与可复核入口

公开仓库为 `mcsaver/oscpu`，机器可读主张表、选择协议、独立复核和构建入口分别由 `claim-evidence-map.tsv`、`case-selection-protocol.md`、`independent-claim-review.tsv` 与 `paper-reproduction.md` 承担，完整相对路径和 `schema` 见附录 C。可版本控制的合同、状态、收据、摘要、索引和哈希直接保留；重型日志或波形由 `manifest` 记录路径、尺寸、SHA-256、时间和 `marker`。原始对象丢失时登记 `MISSING`，受访问限制时登记 `RESTRICTED`，二者均不由摘要替代。

早期 `episode` 缺少统一的模型快照、`seed`、采样参数、完整系统提示、工具序列、上下文截断状态和部分重型原始对象。这些记录仍可说明接口如何演进，但不进入模型级复现、效率比较或严格总体率的分母。由此固定证据协议后，下一章才沿验证压力追踪方法如何从 RV32 局部运行扩展到 RV64 系统事务与 OoO 性能资格化。

9 需求驱动的方法演进与工程里程碑

9.1 验证压力与环境能力的共同演进

早期把“环境建设”和“芯片开发”混在同一条非结构化对话中，会产生三个直接问题：修改 RTL 后无法判断旧日志是否仍属于当前设计；系统启动现象容易被概

括为超过证据层级的结论；反复出现的调试经验仍停留在个人上下文，下一任务只能重新发现。Loop-Work 将二者通过合同接口分开：芯片节点只负责形成当前设计的候选和业务证据，环境节点只负责把已验证且可迁移的规则、Skill、执行入口和证据协议版本化；两者通过有类型的输入输出相连，而不共享模糊的“完成”状态。

这种拆分没有形成两条互不相关的路线。每当处理器任务暴露稳定失效模式，外层闭环就增加一个可复用环境能力；新能力随后缩小下一类任务的可执行范围，并提高其证据要求。表 15 按验证压力列出这种共同演进。它支持“同一接口在更强任务上持续形成可追踪产物”的过程结论，不构成某个环境构件单独提升生产率的因果实验。

Design Conductor 报告的 12 小时 RV32I+Zmmul spec-to-GDSII 案例说明长时间自治已经能够贯穿 RTL、验证、时序优化与后端工具 [25]。本文不把该结果与本仓库的 RV64 OoO 或 Yosys 代理 PPA 作速度、频率或质量对比：两者的 ISA 范围、微架构、工艺、工具与 oracle 均不相同。它在这里承担的作用只是界定外部能力上界；表 15 仍按本案例实际经历的验证压力记录可声明范围，不能用外部端到端成功替代本地系统长跑、身份一致性或物理资格。

附录表 33 进一步追踪方法构件为何进入系统。分析单位不是文件首次出现的时间，而是“哪种失效反复阻断交付、为此在四面之间增加了什么接口、该接口后来被什么任务复用”。这避免把 Git 时间戳误写成因果证据，也使方法演进能够脱离当前目录结构进行复现。

正文保留三条承重过程链：NEMU/Ubuntu 的多节点压力推动“profile 闭包 → 原子证据包”；RV64 跨模块任务推动“架构事实 → 机器合同 → 隔离复核”；十九小时长跑推动“运行状态 → 证据完成位/信号/清理合取 → 限定终态”。它们分别扩展了调度、作用域控制和失败关闭能力；其余构件的触发压力与上下游位置见附录表。

以下里程碑只追踪“工程压力—接口增量—后续调用”的过程链；环境构件的独立生产率效应留给受控实验评价。

9.2 从静态候选到局部动态语义闭合

早期冻结记录包含 RV32I 非流水线 NpcCore 与 8 个辅助 RTL 文件。Verilator lint 通过，但没有 testbench、镜像和参考对拍，因而只能接受“语法与静态结构成立”。随后的动态接通任务把 AM hello image、DPI/C++、pmem、串口和退出码 0 连接起来，并继续增加 itrace、mtrace、dtrace 与批量重放。Agent 的介入由“一次生成候选”变为“生成—运行—观察—修正”，工程师则开始把隐含的完成定义写成可执行接口。

该组证据的有效范围是最小动态闭环，而非完整 ISA、异常或 DiffTest；其方法意义在于把 lint 之后仍缺少的运行接口显式转化为下一轮输入。

表 15: 工程压力、AI 环境演化与可声明里程碑

工程对象与验证压力	暴露的工程盲区	环境标准化与可执行产物	最高可声明范围
RV32 single	lint 无法回答“能否运行”	rv32-bringup 图把 AM image、pmem、DPI、trace、exit 串成动态反馈	给定镜像的最小动态闭环
RV32 SoC	地址图、设备与 target/reference 身份混淆	npc/sim backend 状态、SoC profile、NEMU reference 与 DiffTest 接线	给定测试集上的 SoC 仿真一致性
RV64 target/reference	宽度、CSR、特权和配置变化使旧证据失效	task contract 绑定 RTL/spec/TB/config; 40/40 与 CoreMark DiffTest 分层留证	指定配置的 target/reference 一致性
OpenSBI/Linux/Ubuntu	固件 handoff、rootfs mount、shell 与关机容易被同称为“启动”	Linux 证据八层、virtio/rootfs 合同、long-run status 与 frozen replay	指定 design-id 的系统事务
OoO architecture	owner、恢复、内存序跨模块，局部 PASS 易越级	interface-first、RTL 拓扑、debt ledger、compile-success mutation、独立 reviewer	指定 cohort 的 ARCH_STABLE
CPI/PPA	counter 可干扰、嵌套等待可重复计数、proxy 易冒充 signoff	measurement contract、lost-slot census、同源 PPA identity 与 qualification status	当前仅 PERF_BASELINE; PPA 未合格

相关记录已经出现有界历史检索、子 Agent、独立复核与任务档案等协作结构，但这些结构尚未同 AM image 构建、target/reference、动态 oracle 和可执行完成门连接成统一数据流。人的主要工作仍包括解释运行输出和决定能否接受；Agent 虽能生成、复核并参与短反馈，却不能依据统一合同独立形成动态语义收据。正是 lint PASS 与运行闭环之间的落差，促使后续把 image build、reference、target、compare 和 record 固化为图节点，并把“没有动态证据就不得宣称功能成立”写成 instruction。第一项环境扩展扩大的是执行与解释最小 CPU 事务的闭合能力，而不是从零开始建立所有 Agent 协作结构。

这一里程碑的证据上限是给定 hello 镜像的最小运行语义；完整 ISA、异常、DiffTest 和 RV64 功能由后续层级继续验收。

9.3 target/reference 闭合：从 RV32 single 到 RV64 SoC

随后，npc/sim 被设计为统一入口，按配置分流 npc/single 与 npc/soc。SoC 数据路径为：

代码 4: RV32 SoC 的 target/reference 数据流

```
am-kernels -> AbstractMachine(riscv32-ysyxsoc image)
-> npc/sim(BACKEND=soc) -> npc/soc -> ysyxSoCFull
                        \-> NEMU(CONFIG_SOC_SIM) reference
-> commit-level DiffTest
```

SoC 接通又暴露出另一类环境问题：同一个“运行测试”动作可能在 NEMU reference、NPC single、NPC SoC 上具有完全不同的地址图和结论。这个经验后来进入 E2E 场景隔离规则：nemu-dev* 展开后只允许 NEMU/software-flow 节点，npc-dev 只允许 NPC/software-flow 节点，而显式 integrated profile 保留跨栈闭环；--validate-profile 与真实 dispatch 共用边界检查。这样，profile 不是把模块分开管理，而是防止一个平台的 PASS 在数据流中被误送给另一个平台。

MROM 入口为 0x20000000，SRAM 为 0x0f000000..0x0f001fff；cpu-tests 形成 39/39 DiffTest，char-test 观察 UART，mem-test 在 PC=0x200002c6 到达 GOOD TRAP。这里的关键产物不是另一个 CPU 文件，而是同一镜像在 target 与 reference 两侧可比较。

随后 NPC 先扩展 PC/GPR/CSR/AXI/DPI 到 RV64，关闭尚未资格化的 cache 与 DiffTest，得到 38/38。NEMU 补齐 RV64 CPU state、CSR 与 RV64I/M/B/C/W reference 后，形成 40/40 和 CoreMark DiffTest。这两个冻结点必须区分：38/38 说明 NPC 在给定自测中运行，40/40 才增加 target/reference 一致性证据。

从 32 位扩展到 64 位还改变了 Agent 的上下文组织方式。PC/GPR/CSR/AXI/DPI 不再适合由一个宽泛任务同时修改，局部 reviewer 也不能只读主 Agent 的摘要。为

此，后来的 RTL task Skill 将 RV64 子任务拆为 read-only-review、implementation、verification 与 ppa-analysis 四类，要求合同显式列出 RTL、spec、TB、evidence、写路径和命令，并让子 Agent 在隔离上下文中工作。它没有减少源码探索；相反，它使多 Agent 能够并行复核不同债务项，同时保证每个结论仍能追溯到具体 module/signal/transaction 与测试配置。

39/39 与 40/40 的证据范围分别绑定各自冻结测试集和配置：SoC 全量、Linux、FPGA、物理芯片与跨位宽性能属于更高层验收。

9.4 系统级闭合：NEMU、NPC 与十九小时 Ubuntu 事务

系统级研究先以 OpenSBI v1.8、内嵌 DTB 和 S-mode mini payload 形成固件交接；mini payload 的“S”与 GOOD TRAP 只证明 M-mode 到 S-mode handoff，不等于 Linux。随后，NEMU reference 先进入 Ubuntu 22.04.5 的非交互 shell，NPC 则到达 2 GiB virtio block、EXT4 rootfs、systemd PID1 和 Ubuntu banner。

如果没有证据分层，以上现象很容易被统一写成“Ubuntu 已启动”。.github/instructions/rv64-linux-bringup.instructions.md 因而把路径拆成 env-built、QEMU reference、OpenSBI handoff、kernel progress、init executed、Ubuntu probe、initramfs shell 与真实 rootfs 八级；.github/instructions/virtio-rootfs.instructions.md 又要求 virtio-mmio register、vring descriptor/avail/used、host block backend、UART/virtio 多源 PLIC、DTB、kernel config、/dev/vda 和 mount 证据同时闭合。rootfs 模板必须按内容寻址复制为 run-owned 工作副本，模板前后 SHA 保持不变，guest 合法写入只能改变副本。instruction 在这里直接改变了日志的解释方式：NEMU shell 与 NPC banner 被保留为不同层级，而不是由 Agent 用更顺耳的自然语言合并。

表 16 展示该 instruction 的承重内容。八级不是八条近义描述，而是八个不同验收谓词；上一级的产物只能作为下一级输入，不能自动继承更强结论。例如，QEMU 通过只证明镜像和启动参数存在可用参考，不证明 NPC target；rootfs 构建完成也不证明 guest 已挂载并执行它；使用 RV64IMAC/LP64 的探针更不能替代正式 RV64GC/LP64D 用户态。

表 16: RV64 Linux bring-up instruction 的八级证据与结论边界

级别	状态名称	必须观察到的对象	仍不能声称
1	环境构建完成	固件、内核、DTB、initramfs/rootfs 的构建身份与哈希	任一模拟器已经执行镜像
2	QEMU 参考通过	同一镜像在 QEMU 上达到约定终点	NEMU/NPC target 已通过
3	OpenSBI 交接	固件启动并进入 S-mode payload 或内核入口	Linux 内核已经持续运行
4	内核推进	Linux 解压、设备枚举或内核阶段 marker 持续前进	init 或 Ubuntu 用户态已执行
5	init 已执行	指定 /init 或 PID1 的可归属输出	正式 Ubuntu rootfs 已挂载
6	Ubuntu 探针可见	Ubuntu 用户态文件、动态链接器或版本探针命中	已获得可交互 shell
7	initramfs shell	shell 命令成功且输出/退出码可核验	真实块设备 rootfs 与持久写已闭合
8	Ubuntu rootfs	/dev/vda、EXT4 mount、systemd/服务、读写回验和终止事务	未覆盖配置、设备、服务和长期稳定性均正确

代码 5: NEMU 与 NPC 两个平台的分层日志节选

```
[NEMU]
virtio_blk virtio0: [vda] 4194304 512-byte logical blocks
VFS: Mounted root (ext4 filesystem) on device 254:0
PRETTY_NAME="Ubuntu 22.04.5 LTS"
[ysyx-rootfs] /bin/sh -c exit=0
[ysyx-rootfs] launching /bin/sh

[NPC]
virtio_blk virtio0: [vda] 4194304 512-byte logical blocks
VFS: Mounted root (ext4 filesystem) on device 254:0
Run /lib/systemd/systemd as init process
systemd 249.11-0ubuntu3.21 running in system mode
Welcome to Ubuntu 22.04.5 LTS
Hostname set to <ysyx-ubuntu2204>
```

NEMU 的里程碑是 shell；该次 NPC 的里程碑是 rootfs mount 与 systemd/banner，二者不能合并为同一结论。随后，NPC A3（第三份、也是首份达到完整终态的冻结系统事务证据）把指定 design-id 推进到：

A3 的 host 流水线墙钟约 19 h 14 min 43 s，simulator 自报约 18 h 48 min 22 s；guest 在 49.720532 s 虚拟时间发起 power down。它完成了 OpenSBI、Linux、2 GiB EXT4、Ubuntu systemd、rootfs 写入/同步/回读、virtio direct read/IRQ 增长、有序关机与 clean exit。旧 checker 却把 benign 的 printk: debug: 中 bug: 误识别为 BUG:，所以 source 状态保持 FAIL rc=1，strict 16/17。修正后的 oracle 对冻结 console 重放为 PASS，

代码 6: NPC A3 Ubuntu 22.04 系统事务的关键 marker

```
OpenSBI v1.8
Linux version 6.6.0
virtio_blk virtio0: [vda] ... 2.00 GiB
VFS: Mounted root (ext4 filesystem) on device 254:0
systemd 249.11-0ubuntu3.21 running in system mode
__NPC_CHECK_PASS__:rootfs-write-sync-readback
reboot: Power down
npc: HIT GOOD TRAP
exit via system-reset, code=0,
    cycles=5071521696, commits=1223536213
CPI (cycles/instruction) = 4.145
```

但不改写原始 FAIL，也不授权新 RTL、architecture 或 PPA 晋级。这是 Loop-Work 中“DUT 终态”和“oracle 状态”分离的典型产物。

十九小时运行也反向改变了 AI 环境。短任务可以把进程退出近似看作完成，系统长跑却必须区分 execution/terminal/artifact/assertion/oracle/replay 六种状态；否则一次 HUP、前台 timeout 或 checker 修改都可能把不完整任务留成绿色。由此形成的 task-run-status.sh 显式维护 evidence-complete latch、stage、signal 与 cleanup，agent-system 的 fail-closed 节点再用 early exit、cleanup failure 和 signal 定向测试守住该实现。这项环境能力随后不只服务 A3，也成为所有长仿真、综合、STA 与系统回放的公共交付接口。

NEMU shell、NPC banner 与 A3 系统事务分别属于不同证据层；A3 replay 只重新解释同一冻结观测，设备覆盖、长期稳定、架构晋级和 PPA 资格仍由各自门禁决定。

9.5 跨模块事务合同：排空、FENCE 与 memory owner

OooPendingDrainResolveGate.v 的三个不可变快照展示了工程合同如何从隐含条件变成可验证 owner 边界。

这一演化由两层环境共同约束。接口 instruction 先要求回答“谁产生 stop、谁保持 outstanding、哪类 flush 可以清什么、已发 AXI 和 committed store 是否仍有 owner”；RTL instruction 再要求把需求、状态机、不变量、数据通路与时拍优先级写清。具体复核节点则由 task Skill 限定到该 gate、CoreGlue/ControlPlane 连接、相关 testbench 与证据路径，并保留 scope extension。于是 Agent 不是看到一个布尔表达式后继续叠加条件，而是必须解释每个条件对应的 transaction lifecycle 与反例。

代码 7: 快照 A: 基本后端排空

```
assign backend_drained_o = (rob_count_i == 0) &&
    (issue_count_i == 0) &&
    (core_retire_count_i == 0) &&
    !synth_lane1_ret_pending_i &&
    !synth_lane1_branch_drop_pending_i;
```

快照 A 将 ROB、issue queue、当拍退休和 synthetic lane1 pending 合取为 drain; 它没有显式表示退休 store 仍驻留 SQ 的情况, 也没有完整 memory owner 静默条件。快照 B 增加 mem_retire_quiet_i, 并根据合法状态域证明 rob_count==0 蕴含 retire_count==0, 删除冗余条件:

代码 8: 快照 B: 退休/SQ 静默与冗余条件消除

```
// commit0_fire -> count_q != 0
// commit1_fire -> commit0_fire
// therefore rob_count == 0 -> retire_count == 0
assign backend_drained_o = (rob_count_i == 0) &&
    (issue_count_i == 0) &&
    !synth_lane1_ret_pending_i &&
    !synth_lane1_branch_drop_pending_i &&
    mem_retire_quiet_i;
```

该切片对 16-entry ROB 的有限二值合法域枚举 2176 个用例。枚举器不是把随机输入跑 2176 次, 而是先用 commit1_fire -> commit0_fire、commit0_fire -> count_q!=0 和 commit valid=commit fire 排除不可达组合, 再比较删除前后谓词。附录表 34 给出其中五类代表状态; 正文只保留其语义边界: ROB 空、head 阻塞、单退休、双退休和“lane1 独立退休”这一被排除的非法组合。

在这个声明的有限域内, 原谓词与删除 core_retire_count_i 后的谓词逐例等价; 删除 count guard 和在顶层增加 || 1'b1 的两个故障变体则被拒绝。因此证据支持“该项在枚举域内冗余且检查器对两种旁路敏感”, 不等于完整形式化验证, 也不覆盖 ROB 数据域、所有时序轨迹或全核状态空间。快照 C 再为 ordinary FENCE 增加完整 memory owner graph 静默, 但不把相同条件施加到所有 system event:

代码 9: 快照 C: ordinary FENCE 等待完整 memory owner 静默

```
wire pending_fence_mem_quiet_w =
    !pending_system_fence_i || mem_idle_i;

assign drain_complete_o =
    stop_pending_i && backend_drained_o &&
    pending_control_ready_i && !pending_replay_wait_o &&
    pending_fence_mem_quiet_w;
```

两个可编译负向 RTL 分别删除 FENCE full-memory-idle 谓词和常量化 CoreGlue 到 ControlPlane 的 mem_idle 连接, 均被唯一目标 marker 拒绝; 正向 focused 2/2、module 109/109、validator 12/12。此时只能声明 FENCE-G1=CLOSED; 历史 full-core 仍为 GAP、PPA 为 UNQUALIFIED。三快照之间同时存在其他 RTL 变化, 因此它们证明合同显式化的过程, 不构成单变量性能实验。

代码 10 保留了两个变体的关键 shell 转录。二者都先通过 Verilog 编译, 随后分别在“MIQ/bridge reservation 尚未空闲”和“CoreGlue 连接不再传递真实 mem_idle”的定向场景中被拒绝, 外层 make 均返回 2。由此可以排除“变体只是语法错误”的替代解释; 唯一失败 marker 又把拒绝归因到目标合同, 而不是无关回归。它仍只证明这两个 fault model 可见。

代码 10: 两个编译成功的 FENCE 负向 RTL 及其动态拒绝记录 (删节)

```
[fence_full_memory_idle_removed]
[COMPILE] iverilog ... tb_ooo_pending_drain_resolve_gate
[CHECK-FAIL] fence waits for MIQ bridge reservation idle got=1 expected=0
[FAIL] tb_ooo_pending_drain_resolve_gate errors=1
[RESULT] FAIL status=1

[core_glue_fence_mem_idle_binding_constantized]
[COMPILE] iverilog ... tb_ooo_priv_system
[CHECK-FAIL] fence control plane consumes core memory idle got=0 expected=1
[FENCE-G1-PROGRAM] ... mem_idle_binding=0 ... FAIL
[FAIL] tb_ooo_priv_system errors=1
[RESULT] FAIL status=1
```

2176 个合法域用例与两个 FENCE mutant 覆盖声明的有限状态域和 fault model; FENCE-G1 CLOSED 是 full-core 闭合的输入之一, 三快照则作为合同演化证据而非单变量性能实验。

9.6 反例驱动的候选淘汰与搜索空间收缩

一次双 memory 放宽使同配置 CPI 从 0.645 退化到 0.665, 候选被回滚; branch+ret 虽通过 focused test, Verilator 随后报告 UNOPTFLAT 组合环, 同样回滚。打开分支推测后, 局部 ROB-walk 测试可以通过, 但全核形成“branch speculation 冻结 commit—JALR 等待 backend drain—drain 等待 ROB 排空”的环形等待。spec-on 的 riscv-tests 为 255 PASS/16 FAIL、AM 为 25 PASS/32 FAIL; 回退 mode=0 后 riscv-tests 271 PASS/0 FAIL、module 113/113 PASS。

同一反馈纪律也出现在 NEMU, 而不只在 NPC RTL。nemu-ubuntu-profile 将 basic block、wide ifetch、decode cache、host-vaddr fast path、Sv39 TLB 和诊断 counter

做成显式 runtime flags; detail counter 默认关闭且默认 run 要求其为 0, 从环境层保证“用于定位的探针”不自动成为性能基线。一个 64K decode-cache detail run 将 hit rate 提到 94.52%, 但得到 43.19 MIPS, 低于同容量 default run 的 43.57 MIPS, 工作区据此保留“容量 footprint 抵消 miss 减少”的负结论; 只提前识别 c.sdsp/c.ldsp 的 RVC stack fast path 为 44.31 MIPS, 也低于相邻 baseline 46.95 MIPS, 候选撤回。这些 host MIPS 受 WSL 波动影响, 不能跨 run 任取峰值作稳定收益; 但 profile 的开关、counter 非干扰和相邻反例使“继续扩大 cache/只优化单条压缩指令”成为可审计的被拒方向。这里, E2E 既调度真实系统 workload, 又把负实验转成下一轮 Agent 不应重复的搜索边界。

在后续定向回归中, IFU、FENCE、STORE/AMO 共 23 个 compile-success 负向 RTL 变体被判定器动态拒绝 (19+2+2)。这些结果不证明全状态空间正确, 却证明在指定 fault set 内, 测试不是只能对当前正确版本打印 PASS。对工程师而言, 回滚是一项带复验证据的候选淘汰结果; 对 Agent 而言, 它把下一轮搜索限制在仍然满足基线合同的空间内。

这些回滚把给定配置下的不合格方向从后续搜索空间中排除; 保留基线的全局最优性和未建模回归仍是独立问题。

9.7 性能基线、诊断与未晋级边界

指定产品 cohort full-core-single-hart-rv64-dual-issue-ooo-v1 的 current design-id 为 sha256:093c2380...1a7488。当前收据将该对象标记为 ARCH_STABLE 与 PERF_BASELINE, 但明确写入 PPA 状态 UNQUALIFIED 和 promotion_eligible=false。

表 17: 当前可重复性能基线

workload	cycles	retired	CPI	IPC	资格边界
CoreMark10	5,392,187	3,183,617	1.693730	0.590413	三次 bit-exact, 另有 stats-off 非干扰对照
Dhrystone10000	10,311,431	4,250,000	2.426219	0.412164	三次 bit-exact, 另有 stats-off 非干扰对照

这组数字不是从两个成功日志中直接抄出。A1 与 A2 的原始 performance-baseline-current.status 均仍为 FAIL, 字段分别为 rc=1、stage=evidence-complete、evidence_complete=0 与 cleanup_rc=0; 当前 PERF_BASELINE 是独立的 endpoint-corrected composite replay 收据, 显式引用两份原始 FAIL、A2 command status、run manifest、measurement contract、counter schema 和 ARCH_STABLE pre/post flight。前述 v14n reviewer 合同正是围绕这些对象生成, 要求审查者重新核算 lane endpoint 与 slot 守恒, 并确认 replay 没有改写原状态。这里, Skill 约束审查输入, performance

instruction 约束判定, receipt/schema 约束可消费结果, 三者共同形成了表 17, 而不是由 Agent 自行选择一个更好看的 CPI。

CPI census 显示两个 workload 的 memory latency、request outstanding 与 AXI write response 分别为 CoreMark 的 2,542,049/1,139,006/691,538 cycles, 以及 Dhrystone 的 7,100,359/4,510,140/2,969,998 cycles。这些计数是嵌套驻留, 不能相加成总周期; 它们只形成竞争假设。

为进一步区分事务阶段, NpcOooOwnerTimingProbe.sv 通过 SystemVerilog bind 接入只读观测逻辑, production manifest 的 148 个文件和聚合 SHA 在观测前后不变; 9 个 C++ test、SV lint 和完整 DPI link 通过。然而六次 diagnostic benchmark 虽都正常结束, 三次 CoreMark 均得到 complete=0, invalid=1176, 三次 Dhrystone 均为 complete=0, invalid=49993。收据因此在 receipt-build 阶段 FAIL, 不授权 production RTL candidate。这个结果体现了本文方法的关键价值: benchmark 可以完成, 诊断设施可以链接, 但身份不守恒的量测仍不能成为优化依据。

Yosys 曾给出 18,115 wires、296,118 bits、14,645 generic cells 和 24.82 s; OpenSTA 5 ns proxy 的 top40 为 40/40 MET, 最差裕量 +0.017907454 ns。但该报告有 303 个输入无 input delay、1873 个输出无 output delay、1875 个 endpoint 未约束, 并缺少 SPEF、CTS、OCV、真实宏和多 corner。因此这些数字只能用于结构筛选, 不能写成 200 MHz、面积或功耗签核。

因此当前成熟度停在 PERF_BASELINE: owner-timing 的 FAIL 阻断瓶颈归因, Yosys/OpenSTA 结果只承担结构筛选, PPA 晋级仍保持 UNQUALIFIED。

这些里程碑给出了方法在不同验证压力下的具体实例。据此, 工程产物可以按可迁移构件和 P1-P4 的当前观察加以综合, 而无需再次复述时间线。

10 工程成果与证据综合

10.1 可迁移的工程构件

工业 RTL 项目通常由架构、前端、验证、性能、后端和发布角色共同推进。单个 Agent 回答很难成为这些角色的稳定接口; 身份绑定的合同与证据包则可以。本文方法提供六类可复用工程构件:

在这种组织中, instruction、Skill 和 E2E 本身也是可版本化工程资产, 而不再是某位工程师账号中的个人提示词。commercial-delivery-readiness 节点已经能构建相对路径包, 核对三层 policy、Skill、维护脚本和数据库入口, 排除目录重复嵌套, 并扫描本机 home、Windows 用户路径和本地标识。该节点 PASS 只表示 AI 开发环境达到当前 packaging contract, 不代表处理器或组织流程获得认证; 但它说明同一套方

表 18: 方法构件、工业问题与可交付产物

构件	对应问题	直接产物	当前证据边界
主张编译器	“完成”在不同角色间含义 漂移	任务分类、节点合同、成功条件	尚未形成跨组织标准
分层实验身份	不同设计、工具或 workload 串证	清单与 design、build、workload、evidence 四类身份	历史早期记录不完整
反馈路由器	所有 FAIL 都被错误地导向 RTL 修改	根因/竞争假设、冻结重放、回滚与证据缺口	依赖可观察性与判定器质量
负向资格化	当前 PASS 无法证明测试敏感	编译成功的负向 RTL 与唯一拒绝 marker	只覆盖声明的故障集合
失败关闭发布	长跑中断或证据缺失仍遗留 绿色状态	stage、signal、cleanup 与证据完成收据	不替代功能或物理签核
成熟度阶梯	局部功能、系统与 PPA 结论 相互越级	ARCH_STABLE、PERF、PPA、SYSTEM 状态	当前不具备晋级资格

法可以把“如何使用 Agent”从口头经验变成可审查、可供迁移验证的内部开发基础设施。

这些构件在机制上允许团队把交接对象从截图和口头结论转为可重建实验，把系统层反例沉淀为定向回归，并在 checker-only 变化时重放冻结证据。本文案例实际观察到的是：旧身份消费、局部 PASS 晋级、不合格 owner 量测和代理 PPA 越级被相应门禁拒绝。由于没有对照组，本文不推断因此节省了多少人时、算力、缺陷逃逸或上市周期。

10.2 P1–P4 的当前观察与证据边界

方法章已经定义 P1–P4 的预测和证伪事件，本节只报告案例中的对应观察，避免把命题定义与工程验证写成同一件事。

这些观察与方法预测一致，并给出清楚的拒绝事件；相对生产率、跨模型恢复成本和跨团队迁移收益仍由后续受控实验回答。

在可迁移构件和 P1–P4 观察之外，本章再按研究问题与描述性指标回收同一批案例证据。案例边界、主张选择、编码冲突与数据保留协议已经在前置的案例研究设计章固定，因此下列统计承担审计责任，不能脱离其选择规则解释为总体成功率。

表 19: P1–P4 在当前案例中的观察与证据上限

命题	当前观察	当前证据上限
P1 身份拒绝	V9O 身份元组冲突被降级；A3 原始 FAIL 与冻结 replay 分开保存和消费	覆盖已记录的错配类型，尚无总体逃逸率
P2 负向资格	IFU、FENCE、STORE/AMO 的 23 个可编译目标变体均被对应门禁拒绝	只说明该 fault set 下的判定器敏感性
P3 失败关闭	A3 原始状态未被重放覆盖；TERM、marker 缺失、cleanup 和 owner 收据失败均未遗留绿色终态	覆盖已注入的终止与发布故障
P4 有界晋级	branch speculation 候选回滚，owner 量测保持 FAIL，代理 PPA 保持 UNQUALIFIED	证明这些个案未越级，跨任务复用阈值仍需系统编码

10.3 研究问题的当前回答

表 20 给出当前案例能够回答到的范围。

表 20: 研究问题、观测材料与当前回答边界

问题	关键观测材料	当前回答	不能推出的结论
RQ1	RV32 lint/动态闭环差异；合同、profile 与收据数据流	自然语言目标可被编译为带对象、动作、门禁和证据责任的交付单元	相对直接提示的平均速度提升
RQ2	长跑故障注入；A3 原始 FAIL/冻结重放；23 个 compile-success mutant；owner 量测拒绝	身份、执行、oracle、mutation 与发布门能够把不同失败导向不同终态	对未建模故障的完备拒绝率
RQ3	single→SoC、RV32→RV64、NEMU/NPC、Ubuntu 与 OoO 里程碑	同一接口在所列 RV64 压力层级上均可实例化，并产生合同、观测和边界状态	尺度验证完成，或跨仓库、跨团队的普遍可迁移性
RQ4	PERF_BASELINE、CPI census、owner probe FAIL 与 PPA UNQUALIFIED	功能前置门、量测合同和成熟度状态可阻止诊断值越级为晋级值	已实现稳定 CPI 优化或物理 PPA signoff

10.4 当前 30 条主张的描述性审计

docs/research-paper/claim-evidence-map.tsv 的冻结 SHA-256 为 f20a63ce701d31703f9fe3615d9bb7ae45c134b8b681293c1d98515facf7ab7a。30 条主张中有 18 条 SUPPORTED、4 条 PARTIALLY_SUPPORTED、1 条 OBSERVATIONAL_ONLY、

6 条 CONTRADICTED、1 条 MISSING。CONTRADICTED 不等于“系统失败六次”：其中包括论文主动构造并由现有证据反驳的强命题，例如“代理 STA 已证明 200 MHz 物理签核”。同理，SUPPORTED 是逐条主张状态，不是方法成功样本。

为把当前审计和未来实验使用同一指标语言，令 $C_i = 1$ 表示 Agent 声明完成， $Y_i = 1$ 表示独立隐藏 oracle 的全部强制门通过， $P_i = 1$ 表示结果已发布或晋级，则严格指标为

$$\text{FCR} = \frac{\sum_i [C_i = 1 \wedge Y_i = 0]}{\sum_i [C_i = 1]},$$

$$\text{VSR} = \frac{\sum_i [Y_i = 1]}{N},$$

$$\text{UPR} = \frac{\sum_i [P_i = 1 \wedge Y_i = 0]}{\sum_i [P_i = 1]}.$$

表 21 把三个指标先用于当前主张表的资格审计。结果不是填入一个看似精确的百分比，而是确认现有字段能否构成合法分子和分母。

表 21: FCR、VSR、UPR 在当前 30 条主张上的资格状态

指标	必需字段与分母	当前材料	当前结论
FCR	真实的完成声明 C 与独立隐藏真值 Y	台账混合原始声明、限定重写和为审计构造的反命题；没有统一隐藏 oracle	UNQUALIFIED； 6 条 CONTRA- DICTED 不得直接作为分子
VSR	每个正式 run 的同预算隐藏验收 Y 与总运行数 N	只有部分事件存在冻结 replay，且未尝试重放的主张没有统一登记	UNQUALIFIED； 不能把 18 条 SUPPORTED 写成 18/30 VSR
UPR	下游消费/晋级位 P 与对应隐藏真值 Y	若干关键事件明确保持 GAP/UNQUALIFIED，但 30 行没有统一的 publication-consumed 字段	UNQUALIFIED； 不能用“未晋级 个案”推断总体 UPR

这个负结果本身完成了描述性审计：当前材料足以给出 30 条证据状态，却不足以估计严格 FCR/VSR/UPR。后文的受控实验设计将 C, Y, P 设为逐 run 必填字段；在此之前，本文只报告 A3 原始 FAIL、owner-timing 失败关闭和 PPA UNQUALIFIED 等可逐项复核的拒绝事件，不把它们汇总为因果效果。

10.5 尚未执行的 B0-B3 受控对照与组件消融

现有纵向材料用于检验接口能否实例化，尚不能识别 Loop-Work 相对直接提示或一般工具 Agent 的因果增益。后续实验因此预注册四个递增条件：B0 为冻结提示的一次生成，B1 加入受预算约束的工具反馈与迭代，B2 再加入有界召回、任务合同和显式成功条件，B3 最后加入实验身份、证据新鲜度、mutation、rollback 与原子发布。任务按局部功能、跨层逃逸、性能回退、身份错误、半发布和控制事务等风险分层，在同一模型、预算、初始提交和执行后端下随机分配；隐藏 oracle 与 mutation 集由被测工作流之外的过程预先冻结。

B0-B3 只估计逐级组合包。P1-P4 的单项归因还需要在 B3 上分别移除身份、负向资格、状态提交或晋级约束，并保持其余组件不变。正式样本量、主要指标 FCR、次要指标 VSR/UPR、盲审、固定停止规则和缺失值处理将在不观察正式结果的前提下预注册；当前 512 次仅是资源规划值。完整实验矩阵已经保存在公开的 `experiment-plan.md`，本文不把计划写成已完成结果。

10.6 构念、内部、外部、结论与复现效度

1. **构念效度**：证据完整不等于 RTL 正确，拒绝假 PASS 也不等于提高最终功能成功率；ARCH_STABLE、PERF_BASELINE 和 PPA 资格是不同构念。FCR/VSR/UPR 在缺少 *C, Y, P* 字段时保持 UNQUALIFIED，避免用主张状态替代隐藏真值。
2. **内部效度**：模型、任务难度、RTL、验证平台和开发者经验同时演进。A3、mutation、rollback 与 owner FAIL 能建立诊断性过程链，但不能识别 Loop-Work 的独立因果贡献；每个里程碑只说明相应接口在该压力层级被实例化。
3. **外部效度**：当前材料来自一个 RISC-V 仓库和一位主要开发者。NEMU、NPC、Yosys 与 OpenSTA 增加了后端和压力类型差异，却不构成跨团队、跨 HDL 或商业项目复制。
4. **结论效度**：30 条主张为目的性审计集，12 条复核由非盲第二 Agent 完成，既不是随机样本也不是人类双编码。状态比例只能描述本表，不能外推总体成功率；未来 512 次仅为初始资源设计目标，正式样本量以预注册功效分析为准。
5. **判定器与系统边界**：mutation 只覆盖声明且非等价的 fault set；A3 只覆盖指定 design/config/rootfs 与系统事务；当前 PPA 缺少完整 I/O、SPEF、CTS、OCV、真实 memory macro、power activity 和多 PVT，统一保持 UNQUALIFIED。
6. **复现与维护效度**：工程证据和冻结 oracle 可以重放，但早期模型、seed、temperature、上下文及部分 raw payload 不完整，商业模型采样轨迹不能重建。合同、

checker、identity 与索引还需要持续维护；当前没有跨项目成本分母，不能宣称其净收益。

因而，当前案例研究回答的是“Loop-Work 能否把特定工程状态转化为可审计主张，以及现有记录支持哪些有限命题”；B0-B3 及单组件消融才用于回答“这些构件是否导致更低假完成率或更高可交付率”。两类证据承担不同的推断责任。

11 未闭合工程目标如何继续约束 Workflow

RV64 双发射完整 OoO 核的未闭合目标被写成一组后续 Workflow 义务：架构债务识别与闭合、历史缺陷回归下沉、architecture gate、arch-stable 冻结、performance measurement contract、CPI baseline、bottleneck census、单机制优化、CPI/PPA 联合资格、系统再认证和可重复 promotion。这里列出的不是既成成果或项目进度，而是对方法的下一组可证伪条件：若 Workflow 无法使每次裁决绑定当前身份、使跨轮对象满足版本与 Ext_0 约束、阻止越级或为候选产生可复核反例，它就不能承载下一阶段决策。

当前状态可以精确写成：指定 single-hart RV64 dual-issue OoO cohort 已有 ARCH_STABLE 和 PERF_BASELINE 收据；当前基线具有 CoreMark/Dhrystone 重复性和 stats-off 非干扰对照；owner-timing workload 收据因非法身份事件失败关闭；PPA 仍 UNQUALIFIED，promotion_eligible=false。因此下一步不是根据驻留排序直接改 RTL，而是先闭合 transaction owner 量测、形成可证伪瓶颈、冻结单机制 candidate 与 guardrail，再进行同身份 CPI/PPA A/B。若修改周期、资源或调度合同，必须重新经过 architecture gate。

该约束还要求把 Linux 首次发现的系统层缺陷下沉到最低充分验证层：软件可见语义优先进入 AM、架构测试或 DiffTest；逐周期 owner、flush、replay、completion/commit 合同优先进入 TB 与 assertion；只有设备组合、系统事务与系统性能继续保留在 Linux/Ubuntu 层。如此，Loop-Work 的反馈不仅修复当前候选，还能形成供后续任务复用的低层检查入口；这些入口能否降低总体验证成本，仍需同预算对照实验。

12 结论

本文提出 Loop-Work，把处理器 Agent 的工程能力定义为在固定验收标准下安全闭合任务的能力。控制面把目标编译为对象、权限和门禁，知识面提供当前任务适用的规则与程序化能力，执行面调用真实编译、仿真、综合和回放工具，证据面将观测绑定到实验身份、判定器和允许声明。内层闭环处理当前候选，外层闭环把重复反例

固化为规则、Skill、checker、合同或新节点；二者共同把一次生成转换为可恢复、可审查的工程状态迁移。

该方法在同一仓库中经历了逐级增强的验证压力：RV32 从 lint 进入动态执行，single 扩展到 SoC，NPC 与 NEMU 建立 target/reference 闭合，OpenSBI/Linux 进入 Ubuntu 22.04 系统事务，RV64 双发射 OoO 核形成 ARCH_STABLE 与 PERF_BASELINE。十九小时 A3 的原始 FAIL 与冻结重放并存，23 个目标 mutant 被拒绝，branch speculation 候选被回滚，owner-timing 收据失败关闭，代理 PPA 保持 UNQUALIFIED。这些结果共同说明，同一四平面接口能够在所列压力层级产生正结果、反例和未晋级状态，而不是只保存最有利的成功日志。

Loop-Work 的工程价值在于把个人提示经验转化为可版本化的团队接口：模型可以变化，任务仍由合同、身份和证据责任约束；项目可以扩展，新的系统反例仍能下沉为低成本检查入口。当前单仓库纵向材料支持方法的可执行性、过程链与声明边界，尚不估计相对其他 Agent 架构的生产率因果增益。后续受控实验将比较提示、工具反馈、合同化闭环和资格化闭环，并用定向消融分别检验 P1–P4。

研究台账与复现材料位于 docs/research-paper/；公开仓库为 mcsaver/oscpu。

参考文献

- [1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, 2024. DOI: 10.52202/079017-1601.
- [2] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation,” in *2023 IEEE/ACM International Conference on Computer Aided Design*, pp. 1–8, 2023. DOI: 10.1109/ICCAD57390.2023.10323812.
- [3] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, “ChatEDA: A Large Language Model Powered Autonomous Agent for EDA,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 3184–3197, 2024. DOI: 10.1109/TCAD.2024.3383347.
- [4] J. Wu, Y. Lei, Z. Zhang, X. Meng, D. Yang, P. Li, J. He, and X. Mao, “Mantra: Mutation Testing of Hardware Design Code Based on Real Bugs,” in *2023 60th ACM/IEEE Design Automation Conference*, pp. 1–6, 2023. DOI: 10.1109/DAC56929.2023.10247962.
- [5] D. Gil, J. Gracia, J. C. Baraza, and P. J. Gil, “Study, Comparison and Application of Different VHDL-Based Fault Injection Techniques for the Experimental Validation of a Fault-Tolerant System,” *Microelectronics Journal*, vol. 34, no. 1, pp. 41–51, 2003. DOI: 10.1016/S0026-2692(02)00128-3.

- [6] O. C. Z. Gotel and A. C. W. Finkelstein, “An Analysis of the Requirements Traceability Problem,” in *Proceedings of the First IEEE International Conference on Requirements Engineering*, pp. 94–101, 1994. DOI: 10.1109/ICRE.1994.292398.
- [7] Object Management Group, *Structured Assurance Case Metamodel, Version 2.3*, OMG Document formal/23-05-08, 2023. Official PDF.
- [8] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappsos, “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes,” in *28th USENIX Security Symposium*, pp. 1393–1410, 2019. USENIX paper page.
- [9] SLSA Community, *SLSA Specification, Version 1.2*, 2025. Official specification.
- [10] G. Schermann, J. Cito, P. Leitner, and H. C. Gall, “Towards Quality Gates in Continuous Delivery and Deployment,” in *2016 IEEE 24th International Conference on Program Comprehension*, pp. 1–4, 2016. DOI: 10.1109/ICPC.2016.7503737.
- [11] IEEE and IEC, *IEEE/IEC International Standard–SystemVerilog–Part 2: Universal Verification Methodology Language Reference Manual*, IEEE/IEC Std 62530-2-2023, 2023. DOI: 10.1109/IEEESTD.2023.10287892.
- [12] International Organization for Standardization, *Road Vehicles–Functional Safety–Part 5: Product Development at the Hardware Level*, ISO 26262-5:2018, 2nd ed., 2018. ISO catalogue page.
- [13] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, “MAGE: A Multi-Agent Engine for Automated RTL Code Generation,” in *2025 62nd ACM/IEEE Design Automation Conference*, 2025. DOI: 10.1109/DAC63849.2025.11133191.
- [14] P. Yubeaton, S. Garg, and C. Hegde, “Exploring the Agentic Frontier of Verilog Code Generation,” *arXiv preprint arXiv:2603.19347*, 2026. Primary preprint page.
- [15] N. Pinckney, C. Deng, C.-T. Ho, Y.-D. Tsai, M. Liu, W. Zhou, B. Khailany, and H. Ren, “Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification,” *arXiv preprint arXiv:2506.14074*, 2025. Primary preprint page.
- [16] F. Cui, H. Hou, Z. Luo, C. Yin, and Y. Liang, “HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks,” *arXiv preprint arXiv:2604.14709*, 2026. Primary preprint page.
- [17] Y. Zhao, Z. Wu, B. Yuan, Z. Yu, H. Zhang, W. Ni, C.-T. Ho, H. Ren, and J. Zhao, “PRO-V-R1: Reasoning Enhanced Programming Agent for RTL Verification,” *arXiv preprint arXiv:2506.12200*, 2025. Primary preprint page.
- [18] X. Xin, J. Lou, J. Li, J. Yan, P. Xiao, D. Wu, H. Li, W. Lu, W. Fan, X. Qu, Y. Zhao, M. Yu, Z. Di, and Y. Lin, “GoGoTB: Agentic RTL Verification with Specification-Grounded Coverage Closure,”

accepted to *2026 IEEE/ACM International Conference on Computer-Aided Design* (forthcoming); author manuscript, arXiv:2607.26181, 2026. Author manuscript.

- [19] C. Deng, Z. Yu, G.-T. Liu, N. Pinckney, B. Khailany, and H. Ren, “ACE-RTL: When Agentic Context Evolution Meets RTL-Specialized LLMs,” *arXiv preprint arXiv:2602.10218v2*, 2026. Primary preprint page.
- [20] N. K. Juluru, C. Deng, C. Yu, N. Pinckney, and B. Khailany, “NVIDIA Nemotron 3 Ultra Leads Open Models on Accuracy and Efficiency in Agentic RTL Coding,” *NVIDIA Developer Technical Blog*, 26 July 2026. Official technical release.
- [21] J. Deng, Z. Chen, X. Wei, T. Xing, C. Wen, Q. Sun, and C. Zhuo, “Can AI Agents Really Complete RTL-to-GDS? Lessons from Benchmarking Tool-Interactive EDA Workflows,” *arXiv preprint arXiv:2607.17528v3*, 2026. Primary preprint page.
- [22] R. M. Ghorab, E. Parisi, C. Gutierrez, M. Alberti-Binimelis, M. Moreto, D. Garcia-Gasulla, and G. Kestor, “NotSoTiny: A Large, Living Benchmark for RTL Code Generation,” *arXiv preprint arXiv:2512.20823*, 2025; presented as a Research Manuscript at DAC 2026. Primary preprint page.
- [23] B. Skakov, S. Abduraimov, O. Nurman, and N. Kabylkas, “Firefly: Shedding Light on Verification Gaps Using LLM-Powered Mutation Testing,” in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD*, pp. 1–10, 2025. DOI: 10.1109/MLCAD65511.2025.11189125.
- [24] S. Jasper, M. Luu, E. Pan, A. Tyagi, M. Quinn, J. Hu, and D. K. Houngrinou, “BugGen: A Self-Correcting Multi-Agent LLM Pipeline for Realistic RTL Bug Synthesis,” in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD*, pp. 1–9, 2025. DOI: 10.1109/MLCAD65511.2025.11189127.
- [25] The Verkor Team: R. Krishna, S. Krishna, and D. Chin, “Design Conductor: An Agent Autonomously Builds a 1.5 GHz Linux-Capable RISC-V CPU,” *arXiv preprint arXiv:2603.08716*, 2026. Primary preprint page.
- [26] H. Zhang, Z. Yu, C.-T. Ho, H. Ren, B. Khailany, and J. Zhao, “LLM4Cov: Execution-Aware Agentic Learning for High-Coverage Testbench Generation,” accepted to *The 43rd International Conference on Machine Learning*; camera-ready author manuscript, arXiv:2602.16953v3, 2026. Author manuscript.
- [27] Siemens Digital Industries Software, “Siemens Advances Self-Verifying Agentic AI Workflows for Semiconductor and PCB Design,” official corporate product announcement, 26 July 2026. Official industry release.
- [28] R. Berke, “Introducing the LBC-bench Leaderboard: A New Public Benchmark for LLMs in Chip Design,” *Silicon Integration Initiative*, 21 July 2026. Official benchmark announcement.

- [29] L. Moreau and P. Missier, eds., *PROV-DM: The PROV Data Model*, W3C Recommendation, 30 April 2013. Official specification.
- [30] SCSC Assurance Case Working Group, *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, Safety-Critical Systems Club, 2021. Official standard page.
- [31] E. Singh, K. Devarajegowda, S. Simon, R. Schnieder, K. Ganesan, M. R. Fadiheh, D. Stoffel, W. Kunz, C. Barrett, W. Ecker, and S. Mitra, “Symbolic QED Pre-silicon Verification for Automotive Microcontroller Cores: Industrial Case Study,” in *2019 Design, Automation & Test in Europe Conference & Exhibition*, pp. 1000–1005, 2019. DOI: 10.23919/DATE.2019.8715271.
- [32] S. Chattopadhyay, K. Devarajegowda, B. Zhao, F. Lonsing, B. A. D’Agostino, I. Vavelidou, V. D. Bhatt, S. Prebeck, W. Ecker, C. Trippel, C. Barrett, and S. Mitra, “G-QED: Generalized QED Pre-silicon Verification beyond Non-Interfering Hardware Accelerators,” in *2023 60th ACM/IEEE Design Automation Conference*, 2023. DOI: 10.1109/DAC56929.2023.10247903.
- [33] P. Runeson and M. Höst, “Guidelines for Conducting and Reporting Case Study Research in Software Engineering,” *Empirical Software Engineering*, vol. 14, pp. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8.
- [34] D. Beach and R. B. Pedersen, *Process-Tracing Methods: Foundations and Guidelines*, 2nd ed., University of Michigan Press, 2019. DOI: 10.3998/mpub.10072208.
- [35] C. O’Connor and H. Joffe, “Intercoder Reliability in Qualitative Research: Debates and Practical Guidelines,” *International Journal of Qualitative Methods*, vol. 19, 2020. DOI: 10.1177/1609406919899220.

A 参考实现的追踪表

本附录保存参考实现的逐文件、逐字段映射和承重实现节选。正文以四面接口、数据流和判定边界为主；这里的材料用于复核“抽象方法由哪些仓库载体实现”，不把文件数量、代码存在或环境自测结果当作方法有效性证据。

表 22: 四面方法接口与参考实现的上下游映射

主归属面、方法算子与实现载体	上游输入	下游输出	在闭环中的意义
控制面：任务生命周期控制 <code>agent-flow.c/.sh</code>	用户目标、任务类别、显式修改路径、证据指针	<code>generation</code> 、固定门禁、候选/正式终态与有界档案	使对话式工作变成有代次、可失效、可终止的控制过程
执行面：软环境入口 <code>agent-env.sh</code> 、 <code>agent-run.sh</code>	仓库根、工具链安装与场景开关	可追踪的 NEMU、AM、NPC 与 EDA 环境和确定命令入口	使非交互节点不依赖个人 shell 隐式状态
知识面：规则、spec 与 instruction	架构目标、接口、Linux 层级、CPI/PPA 裁决规则	不可省略的工程不变量、任务分类和验证顺序	把工程师的隐性判断变成模型可发现的声明式策略
知识/控制接口：Skill 与合同生成器	专业策略、任务目标、RTL、spec、TB 与 evidence 范围	已校验合同、结构化命令和可派发提示	把“应该怎样做”编译为可重复操作，而非继续扩写 Prompt
控制/执行接口： <code>profile</code> 、 <code>agent-e2e.sh</code> 、 <code>module library</code>	合同、依赖闭包、owner、节点函数	按序节点执行、逐节点日志和状态事件	把单点能力组合为从局部修改到端到端判定的任务图
执行/证据接口： <code>task-run-status.sh</code>	命令返回码、证据完成位、清理返回码与信号	原子 PASS 或带 stage 的 FAIL	阻止“进程退出即工程成功”
证据面： <code>report.sh</code> 、 <code>task-run</code> 、 <code>manifest</code>	节点元组、raw evidence、身份与时间	<code>receipt</code> 、 <code>evidence index</code> 、 <code>marker</code> 、 <code>publication</code>	将一次执行转化为可审计且不能半发布的交付单元
证据/知识接口： <code>dev_memory</code> 与索引 API	稳定 memory、 <code>task-run</code> 摘要、evidence 元数据	<code>profile/focus</code> 约束的 <code>context brief</code> 与历史指针	连接上一轮证据面和下一轮知识面，而不全量回灌历史
外层能力闭环： <code>agent-maintain.sh</code> 与 <code>package builder</code>	控制器、Skill、 <code>profile</code> 、 <code>schema</code> 、 <code>policy</code>	<code>quick/final/release</code> 资格结果与可迁移包	方法自身也经过正负向 gate，避免规则存在但不可执行

表 23: agent-flow 命令与状态产物

命令	动作	主要产物
begin	建立 task、分类、档位和 generation=1	meta、events.log
record	登记显式路径与固定 gate pointer；使旧代次失效	paths.log、gates.log
evidence	登记 PASS/FAIL/GAP 与已存在的 artifact	evidence.tsv
decision	登记 hypothesis、evidence、counterexample 与 rollback	decisions.tsv
reclassify	用户扩展范围时改变 task class 并递增代次	分类轨迹、更新后的 meta
finish --candidate	执行同代次门禁，形成待独立复核候选	CANDIDATE_PASS 摘要
finish	复用同代次合格门禁并正式结束	PASS/BLOCKED 与 none/compact/durable 档案

表 24: 专业知识载体在编译链中的分工

载体	编码的问题	进入下一节点的方式	单独不能证明什么
instruction/spec	架构、接口、Linux 层级、CPI/PPA 全序等不变量	由分类与路径选择，并被合同字段、assertion 或 checker 机械化	文件存在不等于规则被执行
Skill	某类任务的可复用操作、参数、模板和反例出口	生成、校验并渲染合同或工程产物	Skill 自测不等于 DUT 正确
machine contract	本轮对象、路径、命令、owner、成功条件和边界	作为 Agent 节点唯一授权与审查基准	合同完整不等于工具已经运行
profile/E2E	节点依赖、函数、输入输出和端到端完成条件	递归展开闭包并逐节点执行、留证	图可展开不等于业务门通过
oracle/mutation	如何把 raw observation 转为限定裁决	生成正向结果、唯一拒绝 marker 与 receipt	给定 fault set 被杀死不等于全状态空间完备

表 25: RV64 子任务 instruction 中的关键字段及其工程作用

规则字段	机器约束	被阻止的失效
工程域与任务类型	固定为本地 RV64 RTL，并区分只读审查、实现、验证和 PPA 分析	软件问题或写型任务借用只读权限
可读/可写路径	规范化仓库相对路径；写路径必须属于可读路径，审查任务写集为空	越过模块边界修改、依赖未登记文件
规范命令三元组	每项绑定命令、读写模式和固定用途；禁止自由用途文本扩大权限	临时拼接更容易通过的验证命令
必读材料	列出完成节点必需的 instruction、spec、RTL、TB 和证据	只根据主 Agent 摘要作结论
交付与成功条件	规定具体产物、反例出口、未知项和客观验收谓词	被迫给出 PASS，或把自然语言总结当证据
合同路径与哈希	渲染任务说明时绑定同一 JSON 的路径和 SHA-256	派发提示与已审查合同发生静默漂移

表 26: 同一 RV64 合同能力在 Skill 与 E2E 中的接线

接线位置	实际绑定对象	可检验结果
Skill	instruction、策略 JSON、合同生成器与 create/validate/render 入口	合同可生成；越权组合被校验器拒绝；提示绑定合同哈希
E2E profile	节点 rtl-task-contract、实现函数、责任主体，以及规则/Skill/生成器输入	该能力能够被任务图发现并调度，而非孤立脚本
E2E 节点函数	生成器审计、自测、命令行自测和合同外路径 mutation	正向用法可运行，目标越权变体会产生非零结果与独立日志
业务验证	合同列出的 RTL testbench、DiffTest、综合、STA 或 PPA gate	决定被测设计是否合格；不由 Skill/E2E 的环境自测代替

表 27: 执行面主要工程后端、原始产物及其证据用途

后端或工具	在本工程中的角色	执行面直接产生的原始产物	证据面可支持的结论与边界
AM 与 RISC-V 交叉工具链	AM (Abstract Machine, 抽象计算机运行时与平台适配层) 把同一测试程序适配到不同后端; 交叉编译器生成 RISC-V 目标码	带哈希的 ELF/二进制镜像、反汇编、链接布局和构建配置	冻结 workload 身份并保证两端消费同一程序; 仅构建成功不证明程序已在任一处理器上正确运行
NEMU	本工程的 RISC-V 软件指令集模拟器, 在 DiffTest 或系统启动中作为功能参考端	退休指令与架构状态序列、异常/设备事件、参考端控制台和终止状态	为 NPC 的指令级状态比较提供参考, 也可建立 Linux 参考路径; NEMU 自身通过不能证明 RTL target 正确
NPC/Verilator	把本地处理器 RTL 编译为可执行仿真目标, 是被验证的 target 而非参考模型	target 退休流、周期计数、总线与设备事务、波形、控制台日志及终止 marker	支持局部 TB、DiffTest 和 Ubuntu 系统事务; 只有与冻结参考、配置及 oracle 同时闭合后才能形成限定 PASS
Yosys/OpenSTA	分别承担 RTL 逻辑综合与静态时序分析, 观察设计实现结构和时序约束	门级网表、单元/位数统计、关键路径、时序裕量及约束覆盖报告	支持结构筛选和代理时序比较; 缺少真实宏、寄生参数、完整 I/O 与工艺角时, PPA 必须保持 UNQUALIFIED
JDK/Chisel 工具链	在需要 ysyxSoC/Chisel 生成路径时, 把 Scala/Chisel 描述展开为可供后续编译的 RTL	生成的 Verilog/SystemVerilog、SoC wrapper 与生成日志	证明生成入口和接口产物可复现; 不替代生成 RTL 的仿真、综合或接口验收

表 28: 环境自验证图的责任分组而非文件逐行解释

方法责任	代表节点	所验证的边界
启动上下文与场景身份	discovery、tool environment、 backend status	规则、工具和当前 target 可发现，但不证明 RTL 正确
知识/运行物边界	three-layer contract、runtime artifact boundary	active rule、Skill、日志与重型产物没有互相冒充
控制与审查责任	state traceback、 reviewer/inspector、RTL task contract	状态可回溯、实现者不能自证、越界合同被拒绝
证据与交付资格	fail-closed status、delivery readiness、profile index	不完整长跑不能 PASS，能力可打包且确实有调度入口

A.1 正文下沉的故障注入与过程证据

本节保存第 5–7 章中用于复核实现和过程链、但不承担方法定义的完整表格。正文仅保留失败关闭合取、四面传递顺序、反馈路由、触发规则与代表性工程链，以实现明细打断主论证。

表 29: 失败关闭提交协议的定向故障注入结果

注入条件	命令 rc	证据完整	清理成功	信号/写入	实测终态
正常到达证据末端	0	是	是	无	PASS
提前正常退出	0	否	是	无	FAIL, 保留当前 stage
主命令失败	7	否	是	无	FAIL rc=7
清理失败	0	是	否	无	FAIL rc=9
PASS 状态写入失败	0	是	是	write rc=73	回退写 FAIL, 记录 status-write rc
HUP 中断	129	否	未授权	HUP	FAIL signal=HUP
INT 中断	130	否	未授权	INT	FAIL signal=INT
TERM 中断	143	否	未授权	TERM	FAIL signal=TERM

表 30: 复合性能合同在四面之间的端到端传递

步骤	责任面	实现动作	形成的下游对象
根目标归一化	控制面	把“看性能是否可靠”拆成端点、守恒、重复性、非干扰、原始状态和 PPA 资格六项义务	有明确验收谓词的任务图
知识装配	知识面	选择 counter schema、measurement contract、两版冻结 run 与未晋级规则	带 provenance 的有界上下文包
合同编译	控制面	生成 read-only JSON，校验路径、空写集和 canonical 命令 purpose，绑定合同 SHA	可派发且不能口头扩展的 reviewer 节点
隔离复核	执行面	reviewer 在隔离上下文中只运行 rg/sed/sha256sum，重新核算而不继承实现者判断	核算表、反例、unknowns 与原始引用
资格判定	证据面	对照 raw result、守恒式、哈希和资格全序，不允许用 corrected replay 改写原始 FAIL	限定的 PASS、FAIL、GAP 或 UNQUALIFIED receipt
节点终止	控制面	仅当合同范围和证据同时闭合才允许下游消费；技术缺口保持 GAP	可追踪交付或明确未闭合项

表 31: 工程反例、方法增量与后续复用的过程追踪

暴露的失败模式	外层闭环固化的机制	后续复用场景	可观察结果及边界
lint 无法回答镜像能否运行	动态 image/pmem/DPI/trace/exit 图节点	RV32 single 与 SoC 动态执行	从语法推进到给定镜像；不等于完整 ISA
target/reference 与配置易串证	场景 profile、分层实验身份和任务合同	RV64 自测、40/40 DiffTest 与 CoreMark 对拍	能区分自测和对拍；无 prompt-only 对照
完整系统日志被旧 oracle 误判	原始状态/版本化 replay 分离，长跑提交失败关闭	Ubuntu A3 replay、HUP/TERM 与 early-exit 自测	保留 source FAIL 并形成限定重放；未测节省时数
owner 量测身份非法、PPA 物理条件不足	measurement contract、守恒式与 PPA 资格全序	PERF_BASELINE 审查和候选授权	量测保持 FAIL、PPA 保持 UNQUALIFIED

表 32: 同一证据事件对内层与外层闭环的不同触发

证据面观测	当前任务的内层动作	触发外层的附加条件	外层修改对象
单次实现或周期错误	定位首个错误事务、修复并回归	无；当前合同足以处理	不修改环境
多次出现同类接口误判	当前节点返工并保留反例	跨节点重复，且可抽象为稳定不变量	知识面的 instruction/Skill ，必要时控制面合同字段
缺少可执行 oracle	当前任务保持 GAP	同类任务均无法形成机器判定	知识面的 oracle/E2E 与证据面的资格规则
长跑退出码与证据矛盾	停止并保留 stage/signal/cleanup	问题来自公共 runner 终止语义	执行/证据接口的状态协议与故障注入
工具或环境能力缺失	当前节点 BLOCKED 或截断	不是单一主机偶发故障，且存在稳定适配方案	执行适配器、环境发现与对应 E2E
历史上下文污染或预算溢出	缩小当前查询并保持来源	在多次任务中重复引入过期或无关结论	知识面的 bounded recall 与证据索引策略

表 33: 工程压力驱动的方法构件演进及其上下游作用

方法构件	触发压力	新增方法算子	上游/下游位置	扩大的交付能力
图执行与原 子证据	NEMU/Ubuntu 任务已无法由一条命令表达	profile 闭包、逐节点状态和不可半发布的证据包	合同 → 节点图 → 收据	多节点场景可调度、可定位失败并留证
确定环境与 有界召回	非交互环境漂移、历史记录持续增长	工具环境装载、活动索引和受预算约束的历史摘要	环境合同 → 工具；证据索引 → 知识包	新会话可重建必要环境而不吞入全量历史
环境自验证	规则、 Skill 与实现入口可能彼此漂移	静态审计、正例、 mutation 和发布资格	方法资产 → E2E → 版本化能力	AI 环境自身成为受验证产物

表 33: 工程压力驱动的方法构件演进及其上下游作用（续）

方法构件	触发压力	新增方法算子	上游/下游位置	扩大的交付能力
RTL 任务合同	RV64 跨模块任务需要隔离派发与专业复核	作用域、规范命令、交付和验收的机器合同	架构事实 → Skill → 子 Agent/TB/EDA	多 Agent 仍绑定 module、signal、transaction 与写路径
长跑提交协议	十九小时系统运行不能把进程退出等同于成功	证据完成位、stage、signal、hash 与 cleanup 合取	运行状态 → 资格化 → PASS/FAIL	中断、旧判定器与不完整证据不能遗留绿色状态
轻量生命周期控制	高频开发中全量 profile 成本过高且门禁选择易漂移	任务分类、代次失效和显式路径到固定门禁的路由	分类/路径 → 门禁 → 候选/复核/终态	日常任务运行最小责任门集，同时保留正式复核

表 34: 2176个 ROB 合法域用例中的代表组合与期望

ROB/退休条件	合法域约束	期望退休数	检查目的
count_q=0	lane0、lane1 均不得 fire	0	证明 ROB 空时退休数必为 0
count_q>0, head 被阻塞	两条 lane 均不 fire	0	区分“有条目”和“本拍可退休”
lane0 fire, lane1 阻塞	lane1 不得独立越过 lane0	1	覆盖单退休合法路径
count_q>1, 两 lane 均 fire	lane1 fire 蕴含 lane0 fire	2	覆盖双退休合法路径
lane1 fire、lane0 不 fire	违反提交顺序，排除出合法域	不适用	防止用不可达输入伪造反例

A.2 控制状态与门禁路由

代码 11 把“候选变化使旧证据失效”落到代次字段；只有代次、门禁标识和状态同时匹配时才允许复用。代码 12 则示例化路径如何选择固定门禁，避免由执行 Agent 临时拼接更容易通过的判定命令。

代码 11: 任务代次与旧 PASS 失效的承重实现

```
1 typedef struct {
2     char task_id[MAX_ID];
3     char task_class[MAX_CLASS];
4     char archive_mode[16];
5     char status[MAX_STATUS];
6     int64_t started_ms, gate_ms, paused_ms;
7     int generation;
8 } FlowMeta;
9
10 if (touched) {
11     ++meta.generation;
12     snprintf(meta.status, sizeof(meta.status), "ACTIVE");
13     write_meta(state_dir, &meta);
14 }
15
16 if (atoi(raw_generation) == generation &&
17     strcmp(raw_id, gate_id) == 0 &&
18     strcmp(raw_status, "PASS") == 0) {
19     result->reused = 1;
20 }
```

A.3 知识编译与复核合同

代码 13 说明 Skill 如何把规则编译为可校验 JSON；代码 14 给出一个只读性能复核节点的权限、输入和允许结论。二者分别对应通用合同构造与一次具体实例，不能替代 DUT 的功能或性能 oracle。

代码 12: 显式修改路径到固定门禁的实际路由（错误处理从略）

```
1 if (path == "scripts/agent-e2e.sh" ||
2     starts_with(path, "scripts/e2e/") ||
3     starts_with(path, ".github/e2e/"))
4     add_gate(gates, "profile-bindings");
5
6 if (starts_with(path, ".github/skills/"))
7     add_gate(gates, "skill-audit");
8
9 if (starts_with(path, ".github/skills/prepare-rtl-task-contract/") ||
10     path == ".github/instructions/rtl-agent-task-contract.instructions.md")
11     add_gate(gates, "rtl-task-contract");
```

代码 13: Skill 将专业规则编译为 RV64 子任务合同的关键字段

```
1 allowed_paths = unique([
2     *args.allow_path, *required_context, contract_relative_path
3 ])
4 allowed_commands = [{
5     "command": command.strip(),
6     "mode": "read-only",
7     "purpose": purpose_catalog[command.strip()]["purpose"],
8 } for command in args.allow_read_command]
9
10 contract = {
11     "task_kind": args.task_kind,
12     "scope": {"allowed_paths": allowed_paths,
13              "write_paths": unique(args.write_path or []),
14              "allowed_commands": allowed_commands},
15     "context": {"required_files": required_context},
16     "deliverables": unique(args.deliverable),
17     "success_criteria": unique(args.success_criterion),
18 }
```

代码 14: PERF_BASELINE 独立复核合同的承重字段

```
1 task_kind = read-only-review
2 goal = recompute(endpoint, cycle_slot_conservation,
3                 three_run_bit_exact, stats_off_equivalence,
4                 raw_A1_A2_state, PPA_qualification)
5 allowed_paths = {
6     performance-counter-schema-v4.json,
7     performance-baseline-current.json,
8     versioned raw-run v1,
9     versioned corrected-run v2
10 }
11 write_paths = {}
12 allowed_commands = {
13     (rg, read-only, search-allowed-paths),
14     (sed, read-only, view-selected-lines),
15     (sha256sum, read-only, hash-declared-artifact)
16 }
17 acceptable_outputs = {PASS, GAP, inconclusive}
```

B 工程能力层级、执行端与 Agent 角色变化

表 35: 各能力层级的执行端记录与职责重心

能力层级与 证据对象	可核验执行端	Agent 主要动作	工程师主要责任
局部生成与 动态接通: RV32 single target / reference 与 位宽扩展: SoC、RV64	task-report 多记录 GitHub Copilot; 精确 底层模型未统一冻结 商业模型快照未稳定 保存	局部生成、报错解释、RTL 铺开 跨文件候选、Cache/OoO 修 改、定向调试	定义接口、提供上下文、补 建动态验证 冻结基线、比较候选、决定 回滚
系统级事务: Linux/Ubuntu	进入 Codex/Agent 式 环境; 逐 run model ID 仍不完整	跨模块追踪、Linux bring-up、长链调试	建立 profile、有界召回和证 据结构
架构合同: OoO、 FENCE 与 memory owner	Codex/Agent 式环境; 精确模型未统一保存	身份绑定、mutation、冻结 重放、分层验证	定义 fault set、hard gate 和晋 级边界
多目标资格: CPI/PPA	Codex/Agent 式环境; 精确模型仍不完整	基线重放、CPI 分解、外置 探针	冻结 workload/ROI, 拒绝不 合格量测

能力层级以验证压力和方法接口为主。执行端名称只能证明使用表面，不能反推出底层模型版本。早期缺失 provider、model ID、seed、temperature 和完整 context snapshot，因此本文不补造“当时是什么精确模型”。可以确认的是 Agent 的角色随环境成熟而变化：从候选生成器，逐步成为在工程合同内执行、验证、回滚和留证的节点。

使用规模背景。累计使用 227.5 亿 Token，单日峰值 14.6 亿，最长单次聊天为 24 小时 11 分，连续使用 28 天。这些指标只说明交互规模与持续性，并解释为什么上下文边界、有界召回、证据压缩和任务恢复成为实际工程问题；Token 数不等于代码量、正确率、生产率或方法效果。

C 机器可读主张—证据附录

本附录不把 30 条长记录重复排入 PDF，而是固定机器可读对象及其校验值。权威数据文件为 docs/research-paper/claim-evidence-map.tsv，冻结于提交

858fd3cf0dc6cb466c80707051f40d4a4710fcbc; 文件 SHA-256 为 f20a63ce701d31703f9fe3615d9bb7ae45c134b8b681293c1d98515facf7ab7a。首行 schema 按原始顺序为:

代码 15: 主张—证据表的 22 列 schema

```
claim_id, claim_text, research_axis, evidence_status, date,
task_episode, run_id, profile, source_path, raw_evidence_path,
git_commit, design_id, configuration, command, exit_code,
metric, negative_evidence, rollback_or_rejection, claim_boundary,
reproducibility, paper_section, notes
```

这些列承担四类不同责任: claim_id 至 evidence_status 固定主张与编码; date 至 configuration 绑定 episode 和实验身份; command 至 rollback_or_rejection 保存执行、观测与反例; claim_boundary 至 notes 保存外推边界和复现缺口。缺失的 design_id 或 git_commit 在 TSV 中以连字符显式表示, 不由论文补造。

表 36: 冻结主张表的状态分布与解释边界

状态	数量	在本研究中的含义
SUPPORTED	18	对象、来源和边界支持当前限定措辞; 不是方法成功次数
PARTIALLY_SUPPORTED	4	机制或结果存在, 但强度、分母、身份或归因仍不完整
OBSERVATIONAL_ONLY	1	只支持共现或纵向变化, 不支持因果解释
CONTRADICTED	6	原始证据直接否定所编码的强命题; 其中包含主动审计的反命题
MISSING	1	当前公开材料不足以裁决, 不以摘要或路径存在替代证据
合计	30	目的性审计集, 不是随机样本或运行成功率分母

选择与去重规则由 docs/research-paper/case-selection-protocol.md 固定。独立反例审查样本为 C01、C02、C03、C04、C07、C10、C11、C14、C15、C22、C23、C30; 复核结果位于 docs/research-paper/independent-claim-review.tsv。初审阻断 C01、C07、C30, 分别暴露早期阶段被过度简化、同一发布内身份元组冲突和原始路径覆盖不足; 最终裁定保留了措辞收紧、状态降级和证据补齐的轨迹。该复核用于发现转录与边界错误, 不构成人类编码一致性或方法效果证据。

早期候选全集及逐条排除日志没有完整保留, 这是本附录仍不能补偿的选择偏差。因而 TSV 允许读者复算状态分布、追到现有来源并检查边界, 但不能复原所有未入选候选, 也不能计算严格 FCR、VSR 或 UPR。

D 复核入口与复现边界

本文只复核已有工程证据，不复跑生产 RTL、Linux、综合或 STA。工程日志、配置与 design-id 的可重放不等于 Agent 采样轨迹可复现；PPA 代理缺少物理条件时统一保持 UNQUALIFIED。公开材料见 <https://github.com/mcsaver/oscpu>。

表 37: 文章中主要方法与工程证据的复核入口

对象	工作区入口
Workflow 导航与三层 policy	AI_ENVIRONMENT.md、 .github/ai-env/contracts/agent-env-policy.json、.github/instructions/agent-env-layer-contract.instructions.md
instruction/Skill 执行规则	.github/instructions/rtl-generation-workflow.instructions.md、.github/instructions/rv64-ppa-optimization-workflow.instructions.md、 .github/skills/agent-env-maintenance/SKILL.md
轻量任务与代次状态 profile 十一节点闭包与发布实现	scripts/agent-flow.c、scripts/agent-flow.sh .github/e2e/profiles/agent-system.tsv、 .github/e2e/profiles/discovery.tsv、scripts/agent-e2e.sh、 scripts/e2e/lib/report.sh
agent-system 实际完成机制	.github/e2e/profiles/agent-system.tsv、 scripts/e2e/lib/report.sh、 docs/research-paper/evidence-ledger.md
bounded recall 与 evidence API	scripts/dev_memory/api.py、scripts/github_index_db.py、 .github/ai-env/contracts/agent-env-schema-contract.json
RTL 子任务 Skill 与生成器	.github/skills/prepare-rtl-task-contract/SKILL.md、 .github/skills/prepare-rtl-task-contract/scripts/rtl_task_contract.py
PERF_BASELINE 与独立复核边界	npc/rv64/eval/ppa/evidence/performance-baseline-current.json、npc/rv64/design/arch/performance-boundary-qualification-amendment-v1.json
长跑失败关闭	scripts/task-run-status.sh、 scripts/tests/test-task-run-status.sh
里程碑与证据台账	docs/research-paper/evidence-ledger.md
单模块三快照	docs/research-paper/module-evolution-evidence.md
NPC A3 十九小时里程碑	docs/research-paper/ubuntu-19h-milestone-evidence.md
30 条主张映射	docs/research-paper/claim-evidence-map.tsv
构建复现	docs/research-paper/paper-reproduction.md